

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm



Рисунок 2.1 – Логотип среды разработки PyCharm

2.1 Описание языка

Python – это высокоуровневый язык программирования, с помощью которого создают сайты, разрабатывают приложения, автоматизируют процессы анализа или визуализации данных.

Преимущества Python:

- лёгкость освоения: python прост, логичен, имеет понятный синтаксис, поэтому подходит даже для новичков. Для его изучения не нужны знания английского;
- простота визуального воспитания: блоки кода отделяются отступами, за счёт меньшего объёма код также воспринимается проще.
- кроссплатформенность: программы, написанные на Python, могут запускаться и функционировать на всех типах операционных систем;
- скорость разработки: чтобы написать программу на Python нужно значительно меньше кода, чем при разработке, например – на Java;
- универсальность: благодаря интерпретируемости python он используется практически на всех платформах для различных задач – от тестирования до научных исследований. Код удобно писать даже в стандартных текстовых редакторах;

- множество инструментов: стандартные библиотеки Python способны решать даже сложные задачи. Установка дополнительных модулей, созданных для конкретных целей, помогает при разработке специальных проектов;

- масштабируемость: возможность адаптации высокоуровневой логики позволяет проектам, разработанным на Python, масштабироваться и расширяться.

Недостатки Python:

- медленная работа: этот недостаток лишает разработчиков возможности создавать высокопроизводительные проекты только на Python. Необходимо задействовать другие языки программирования. Программное обеспечение работает медленно из-за устаревшего кода Global Interpreter Lock, сохранившимся её с первой версии языка;

- трудность переноса проектов на другие системы: проблема возникает из-за зависимости языка программирования от библиотек;

- ресурсоёмкость: Python не подходит для проектов, требующих больших объёмов памяти [1].

Python является отличным выбором для разработки приложения для тренировки памяти благодаря своей простоте, скорости создания прототипов и богатому набору инструментов: интуитивно понятный синтаксис и низкий порог входа позволяют быстро реализовать базовую логику (создание карточек, тестов, отслеживание прогресса), а обширные библиотеки дают возможность сосредоточиться на методиках развития памяти, а не на технических сложностях; хотя Python может уступать в производительности при высоких нагрузках, для большинства задач такого приложения его скорость работы и масштабируемость будут вполне достаточны, особенно на этапе старта и активной доработки функционала на основе обратной связи пользователей.

2.2 Описание используемой IDE

PyCharm – это программное обеспечение, которое представляет собой интегрированную среду разработки для языка программирования Python.

Использование IDE PyCharm позволяет значительно ускорить разработку программного обеспечения, минимизировать ошибки и облегчить сотрудничество между программистами. Благодаря широкому спектру возможностей, эта интегрированная среда разработки помогает создавать качественное ПО в короткие сроки.

Использование PyCharm предоставляет ряд преимуществ по сравнению с другими средствами разработки. Рассмотрим основные плюсы использования данной IDE для программирования:

- мощные инструменты подсказок и автодополнения, которые позволяют увеличить производительность и качество кода;
- интегрированная система рефакторинга позволяет легко изменять структуру кода, делая его более читаемым и понятным;
- отличная интеграция с инструментами управления версиями, такими как Git, что упрощает работу в команде;
- интуитивно понятный интерфейс, который упрощает навигацию по проекту;
- широкий выбор плагинов для настройки среды под свои потребности;
- интеллектуальный анализ кода поможет выявить потенциальные проблемы и предложить варианты их решения;
- возможность использования отладчика для пошагового анализа выполнения кода;
- поддержка различных фреймворков, библиотек и инструментов для анализа кода.

Все эти особенности делают PyCharm незаменимым инструментом для разработчиков на Python, помогая им создавать высококачественное программное обеспечение быстро и эффективно [2].

2.3 Описание используемых библиотек

tkinter — это стандартная библиотека Python, предназначенная для создания графического пользовательского интерфейса. Она входит в базовую поставку языка, поэтому не требует дополнительной установки и сразу готова к использованию. С её помощью можно создавать оконные приложения, добавлять кнопки, текстовые надписи, поля ввода и другие элементы интерфейса, а также обрабатывать действия пользователя, такие как нажатия мыши или клавиш. Работа программы строится по событийному принципу: после запуска основного цикла приложение ожидает действий пользователя и реагирует на них через заранее заданные функции.

В данном проекте библиотека tkinter была выбрана благодаря своей простоте и доступности. Она позволяет быстро реализовать интерфейс без использования сторонних инструментов, что особенно удобно для разработки небольших приложений, таких как тренажёр памяти. Кроме того, tkinter хорошо интегрируется с остальной логикой программы и предоставляет все необходимые возможности для создания меню, переключения между экранами, настройки параметров и отображения статистики. Благодаря этому удалось реализовать удобный и функциональный интерфейс без лишнего усложнения кода.

Скриншоты работы в выбранной среде приведены на рисунке 2.2

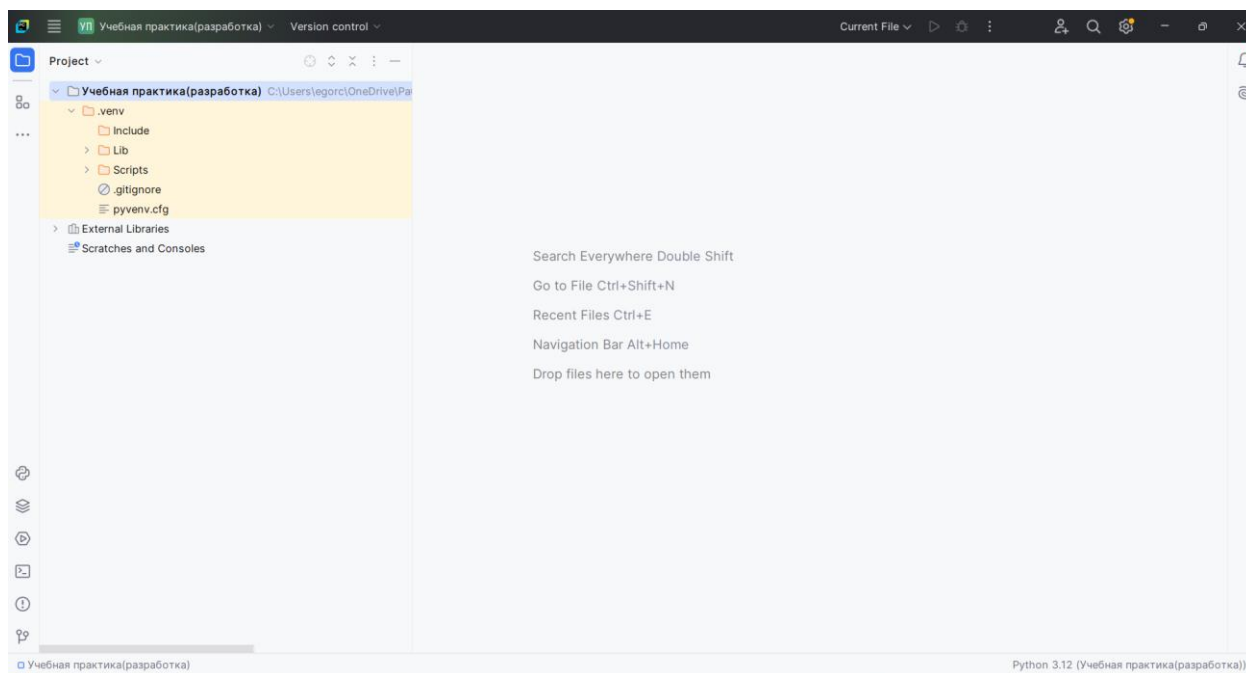


Рисунок 2.2 – Интерфейс среды разработки PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub.

Ссылка на профиль: <https://github.com/ChEgor459>

Ссылка на репозиторий: <https://github.com/ChEgor459/YP.05>

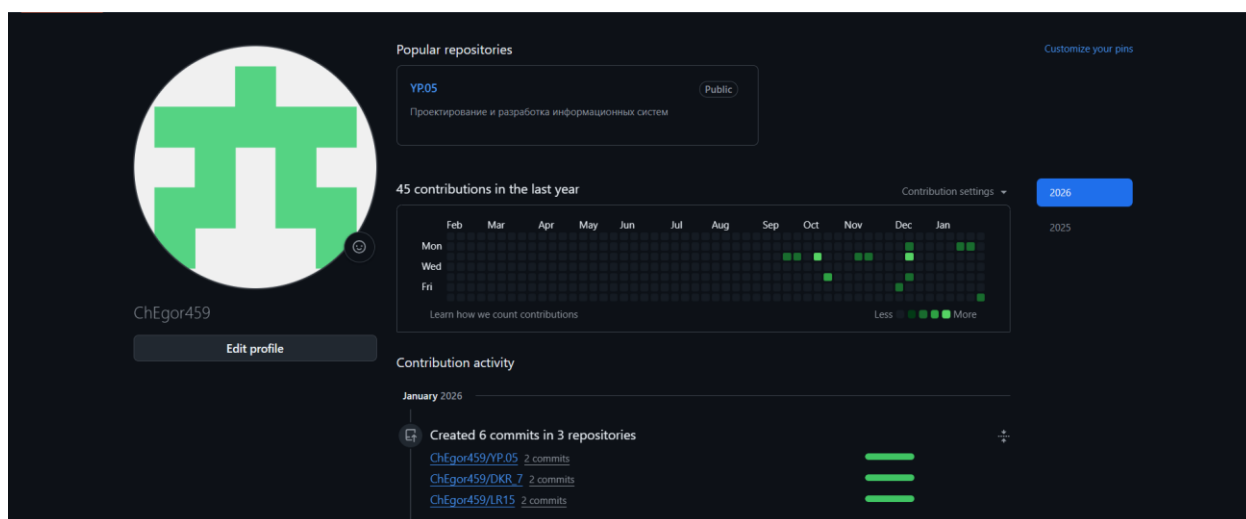


Рисунок 2.3 – Профиль аккаунта в GitHub

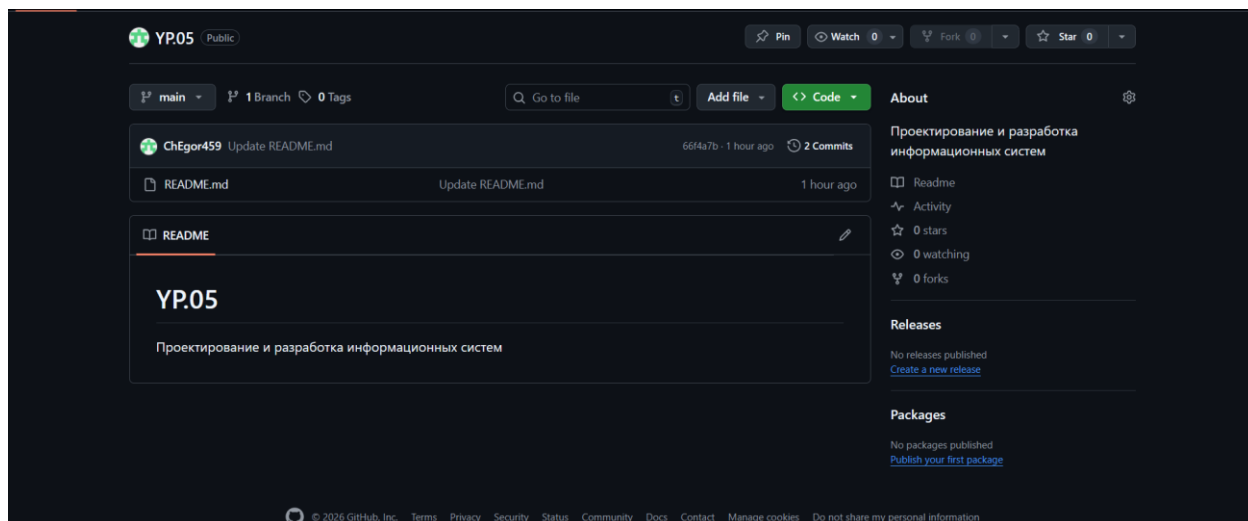


Рисунок 2.4 – Репозиторий для работы

4 Описание выполнения индивидуального задания

4.1 Анализ предметной области

Память человека – это когнитивная функция, а также совокупность процессов, обеспечивающих запоминание, сохранение и воспроизведение информации. Когнитивные способности – это все процессы, посредством которых сенсорная информация преобразуется, сокращается, перерабатывается, сохраняется, восстанавливается и используется. Память формируется в процессе деятельности человека и является основой обучения, мышления и накопления опыта. Изучение памяти началось при исследовании работы мозга и механизмов обработки информации, а также при попытках объяснить, каким образом человек сохраняет и использует полученные знания [3].

Память представляет собой систему взаимосвязанных процессов, включающую сенсорную, кратковременную и долговременную память. Сенсорная память обеспечивает кратковременное удержание информации, поступающей от органов чувств, кратковременная память отвечает за оперативную обработку данных, а долговременная память служит для хранения знаний, умений и опыта [4]. Совместная работа этих уровней обеспечивает целостное восприятие информации и возможность её осознанного использования.

Память не является неизменной – она развивается и перестраивается в процессе обучения и деятельности. Современные исследования показывают, что использование цифровых технологий и гаджетов само по себе не ухудшает память человека, а при правильном использовании может способствовать развитию когнитивных функций, включая запоминание и внимание [5].

Игры для развития памяти представляют собой интерактивные приложения, направленные на тренировку различных видов памяти с помощью специальных заданий. В игровых механиках используются повторение, ассоциативное запоминание, постепенное усложнение и обратная связь, что способствует активации кратковременной и долговременной памяти. Исследования показывают, что видеоигры способны положительно влиять на когнитивные способности человека, включая рабочую память, внимание и скорость обработки информации, при условии умеренного и целенаправленного использования [6].

Таким образом, память является базовым компонентом познавательной деятельности человека, а её развитие с помощью игровых технологий является перспективным направлением в образовательных и развивающих цифровых системах.

Рынок игр и приложений для развития памяти демонстрирует устойчивый рост и значительный пользовательский интерес. По оценкам аналитиков, популярные приложения для тренировки когнитивных функций, такие как Lumosity и аналогичные, имеют от 3 до 10 миллионов загрузок на основных платформах (Google Play, App Store). При этом даже менее известные игры в тематической категории собирают сотни тысяч скачиваний, что указывает на широкую аудиторию пользователей, интересующихся развитием памяти и внимательности.

Показатель удержания пользователей для приложений образовательно-игрового характера, включая тренажёры памяти, в среднем составляет 30–45 % через 30 дней, а через 90 дней – около 15–25 %. Эти усреднённые данные отражают тенденцию, что значительная часть первоначальных загрузок не превращается в долгосрочное использование, однако вовлекающие механики, такие как адаптивные уровни, ежедневные задания и система прогрессии, способны повысить удержание до уровней выше среднего, что положительно сказывается на экономических показателях продукта.

Таким образом, при условии качественного игрового дизайна и стратегии монетизации (freemium с подпиской, внутриигровыми покупками и аналитикой прогресса), игры для развития памяти способны привлекать миллионы скачиваний и удерживать десятки процентов активных пользователей, обеспечивая стабильный доход и долгосрочное присутствие на рынке. Эти усреднённые цифры подтверждают, что разработка подобных приложений является экономически обоснованной и перспективной.

4.1.1 Анализ проблемы

Развитие памяти с помощью цифровых игр является одним из современных и эффективных методов тренировки когнитивных функций человека [1]. Основой данного подхода является подбор игровых заданий, направленных на активацию различных видов памяти, выбор игровой механики, а также настройка уровня сложности в соответствии с возможностями пользователя. Существенное значение имеет структура игрового процесса, обеспечивающая повторение, закрепление и осмысленное воспроизведение информации.

Игры для развития памяти позволяют решать задачу систематической тренировки запоминания, удержания и восстановления информации. В отличие от традиционных методов обучения, игровые технологии обеспечивают высокую мотивацию пользователя и регулярность занятий. Однако игровые приложения для тренировки памяти также имеют ряд ограничений: при неправильном подборе сложности возможно перенапряжение рабочей памяти; однообразие заданий снижает интерес; отсутствие адаптации приводит к снижению эффективности тренировки. Эти недостатки требуют применения адаптивных игровых механизмов, учитывающих индивидуальные особенности пользователя и динамику развития его памяти [4].

Современные исследования показывают, что использование цифровых технологий и видеоигр не приводит к ухудшению памяти, а при правильной организации игрового процесса может способствовать развитию внимания, рабочей памяти и скорости обработки информации [4, 5]. Особенно эффективны игры, в которых задания требуют активного удержания информации, быстрого переключения внимания и восстановления ранее представленных элементов.

Как правило, развитие памяти с помощью игровых приложений происходит по следующему сценарию:

- формирование набора игровых заданий для тренировки различных видов памяти;
- структурирование уровней сложности и последовательности заданий;
- реализация механизма повторения и обратной связи;
- адаптация сложности на основе результатов пользователя;
- анализ эффективности тренировки памяти в процессе использования игры.

Пример игровой механики для развития памяти представлен на рисунке 4.1.



Рисунок 4.1 – Пример игровой механики для развития памяти

Для решения задач развития памяти в игровых приложениях применяется подход моделирования когнитивных процессов. В рамках данного подхода уровень развития памяти пользователя может быть представлен в виде функции, зависящей от времени тренировки и сложности заданий. В результате регулярного выполнения игровых упражнений формируются

устойчивые нейронные связи, обеспечивающие более эффективное запоминание и воспроизведение информации [3].

4.1.2 Обзор аналогов

1. Lumosity

Платформа: Android, iOS, Web

Описание: Lumosity – одно из самых известных приложений для тренировки памяти, внимания и мышления. Оно предлагает пользователю серию мини-игр, в том числе специально разработанных упражнений на запоминание последовательностей, образов и чисел. Игры имеют адаптивную сложность: по мере прохождения задания усложняются.

Преимущества:

- широкий набор упражнений, включая разные виды памяти;
- адаптация под уровень пользователя;
- статистика прогресса и отчёты;
- поддержка веб-версии с синхронизацией.

Недостатки:

- основные функции доступны по платной подписке;
- меньше игрового сюжета, больше тренажёрная направленность;
- повторяющиеся механики со временем снижают интерес.

Пример интерфейса Lumosity показан на рисунке 4.2



Рисунок 4.2 – Интерфейс Lumosity

2. Peak – Brain Games and Training

Платформа: Android, iOS

Описание: Peak предлагает разделы, посвящённые непосредственно тренировке памяти, включая запоминание расположения объектов, чисел, комбинаций. Приложение делает упор на игровой интерфейс с динамичной визуализацией и мотивационными механиками (ежедневные задания, достижения).

Преимущества:

- высокий уровень вовлечённости за счёт игровых элементов;
- разнообразные виды упражнений;
- поддержка ежедневных вызовов и целей.

Недостатки:

- полный доступ – через подписку;
- некоторые пользователи отмечают одинаковость уровней;
- подходит не для всех возрастов из-за скорости подачи.

Пример мини-игры на развитие памяти в Peak показан на рисунке 4.3

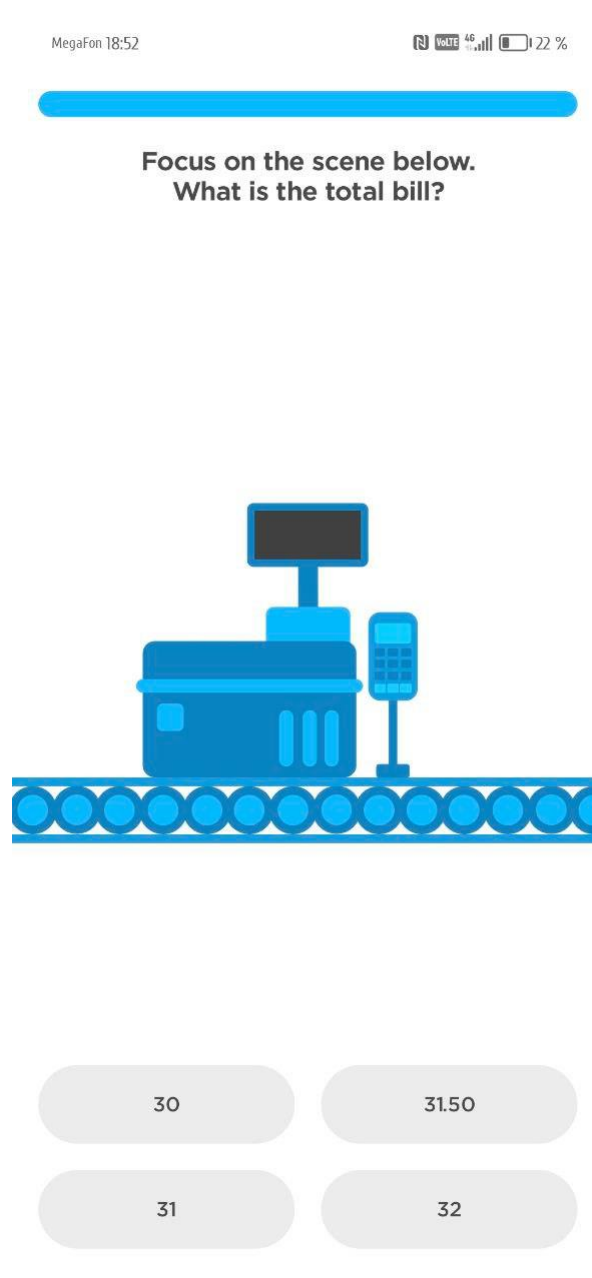


Рисунок 4.3 – мини-игра в Peak

3. Memory Training Game

Платформа: iOS (iPhone, iPad)

Описание: Memory Training Game – мобильное приложение, разработанное для тренировки и укрепления памяти через серию игровых упражнений. Задания включают запоминание расположения, образов и последовательностей с постепенным увеличением сложности. Игра сочетает визуально привлекательный интерфейс с простым и понятным геймплеем, что делает её подходящей для пользователей разных возрастов и уровней подготовки. Пользователь может соревноваться с собственными рекордами и улучшать результаты с каждым прохождением.

Преимущества:

- интуитивно понятный интерфейс, подходящий широкому кругу пользователей;
- нарастающая сложность уровней, стимулирующая развитие памяти;
- поддержка встроенных покупок и отключения рекламы (опция);
- подходит для коротких ежедневных тренировок памяти.

Недостатки:

- ограниченная локализация (англ. интерфейс);
- плохая оптимизация;
- отсутствие глубокой адаптации под индивидуальные особенности памяти;
- основной упор на базовые упражнения, без сложных методик обучения.

Пример мини – игры в Memory Training Game показан на рисунке 4.4



Рисунок 4.4 – мини – игра в Memory Training Game

4. Memory Islands: Brain Match

Платформа: Android

Описание: Memory Islands: Brain Match – это бесплатная игра, сочетающая классические задания на память с соревновательным элементом. В игре пользователь открывает пары карт на виртуальных «островах» и запоминает их расположение, чтобы найти парные изображения быстрее

оппонента. Игра предполагает пошаговые дуэли против других игроков в реальном времени, коллекционирование очков, переход на новые уровни и расширение игровых досок с усложнением заданий. Это делает игру не просто тренажёром, а социальным матчем учебных упражнений памяти

Преимущества:

- интерактивный соревновательный режим с другими игроками;
- постепенное увеличение сложности за счёт больших игровых досок и «островов»;
- элемент коллекционирования очков и прогрессии;
- развитие не только памяти, но и скорости реакции.

Недостатки:

- часто требуется интернет для мультиплеера;
- игровой процесс ориентирован на соревнование, а не на глубокие когнитивные методики.

Пример игрового процесса Memory Islands: Brain Match показан на рисунке 4.5

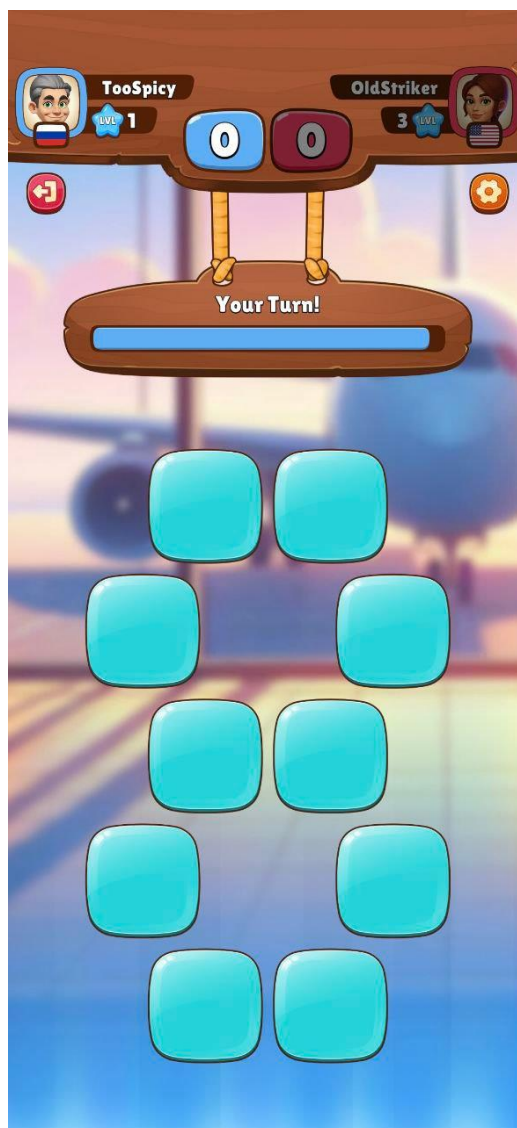


Рисунок 4.5 – игровой процесс Memory Islands: Brain Match

5. Memory Game Master

Платформа: Android (Google Play)

Описание: Memory Game Master – мобильная игра для тренировки памяти, в которой пользователю предлагается находить совпадающие пары, запоминать расположение символов и повышать сложность задач по мере прогресса. Приложение ориентировано на широкую аудиторию от детей до взрослых, стремящихся развивать кратковременную и рабочую память в игровой форме. Игра пользуется стабильной популярностью, имеет около 50

000+ скачиваний и рейтинг ~4,6 из 5 на Google Play, что свидетельствует о положительном восприятии пользователями.

Преимущества:

- игровой процесс построен на классической механике поиска пар, что интуитивно понятно большинству пользователей;
- прогрессия сложности – уровни становятся сложнее по мере прохождения;
- высокий рейтинг пользователей (~4,6), что говорит о качестве игрового опыта;
- бесплатная установка, без обязательной подписки;
- поддержка таблицы рекордов и личных достижений.

Недостатки:

- основной игровой процесс основан на классической механике, что может быть однообразно для части игроков;
- ограниченная глубина тренировок памяти – нет сложных методик или адаптивной персонализации;
- отсутствие сюжетных или социальных механик (соревнования, челленджи с друзьями);
- некоторые пользователи могут считать уровни слишком простыми на начальных этапах.

Пример игрового процесса Memory Game Master показан на рисунке 4.6



Рисунок 4.6 – игровой процесс Memory Game Master

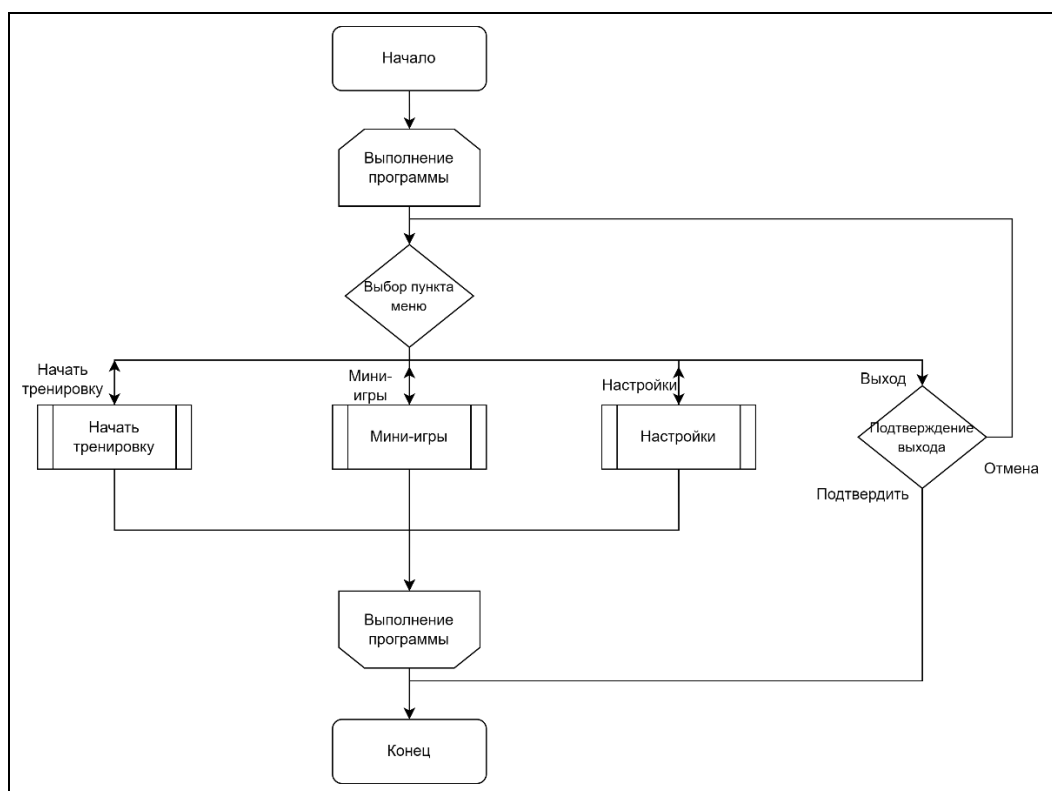
Анализ игр для развития памяти показывает, что большинство приложений используют однотипные мини-игры (поиск пар, запоминание последовательностей, расположение объектов), из-за чего интерес пользователей быстро снижается и возникает высокая конкуренция. Основные недостатки аналогов – слабая геймификация, отсутствие сюжетных элементов, ограниченная адаптация сложности и платный доступ к функционалу. Это подтверждает актуальность разработки игры с более разнообразными

механиками и продуманной системой мотивации, способной удерживать пользователей длительное время.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

После запуска приложения происходит инициализация интерфейса: загружается главное окно со всеми элементами управления и кнопками навигации. Пользователю отображается главный экран с возможностью начать новую игру, перейти к просмотру правил или выйти из приложения. Алгоритм работы главного окна представлен на рисунке 4.2.1.



4.2.1 – Алгоритм работы главного окна приложения для развития памяти

Войдя в режим настроек, пользователь получает доступ к изменению конфигурации приложения: он может выбрать другой язык интерфейса или настроить параметры звука. После завершения редактирования пользователь сохраняет изменения, что инициирует автоматический выход из режима —

возврат на предыдущий экран или в главное меню. Схема данного алгоритма отображена на рисунке 4.2.2.

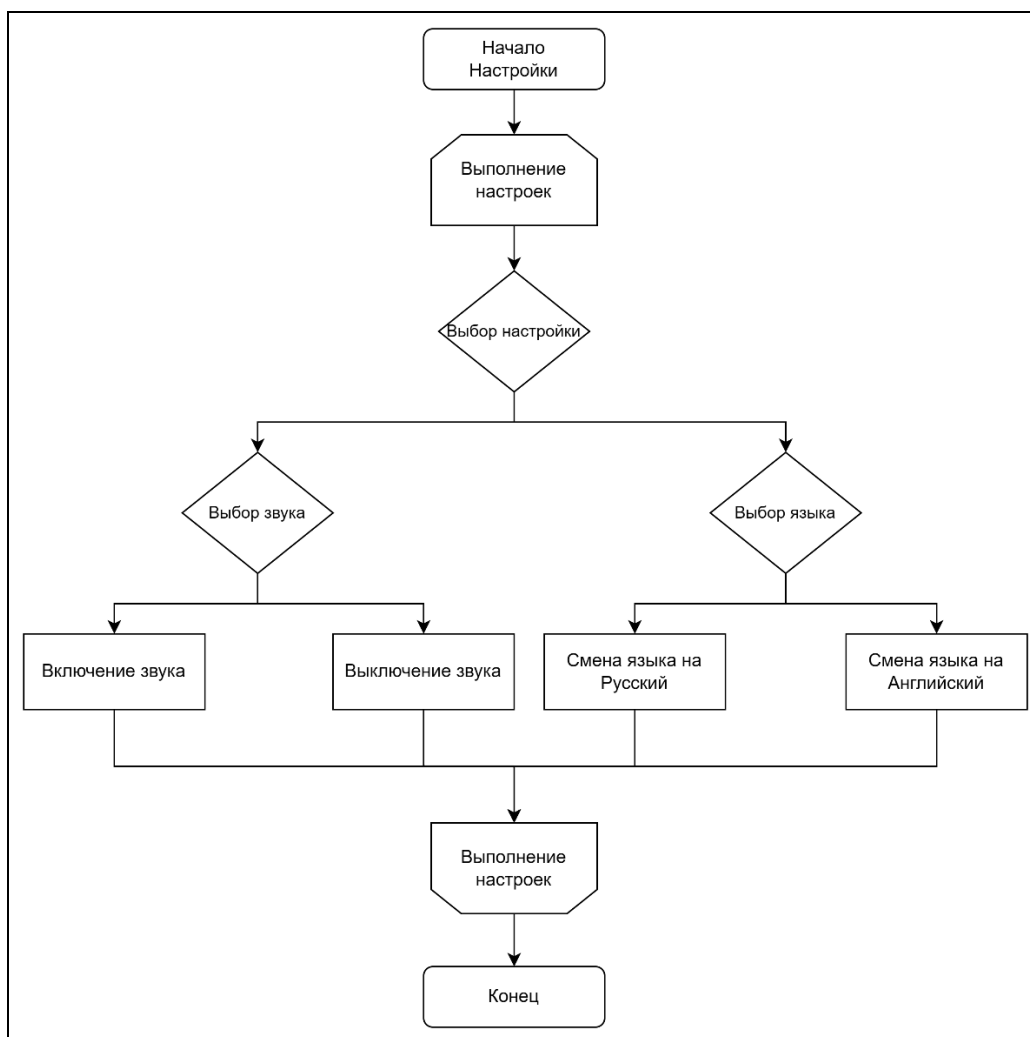


Рисунок 4.2.2 – Алгоритм режима «Настройки»

При выборе пункта «Начать тренировку» пользователю создается игра, составленная из трёх случайных мини-игр, которые нужно будет пройти. После прохождения тренировки происходит переход на экран с результатами. Алгоритм пункта «Начать тренировку» представлен на рисунке 4.2.3.



Рисунок 4.2.3. – Алгоритм пункта «Начать тренировку»

При выборе пункта «Мини-игры» пользователь выбирает мини-игру, которую он хочет пройти. Алгоритм данного пункта представлен на рисунке 4.2.4.

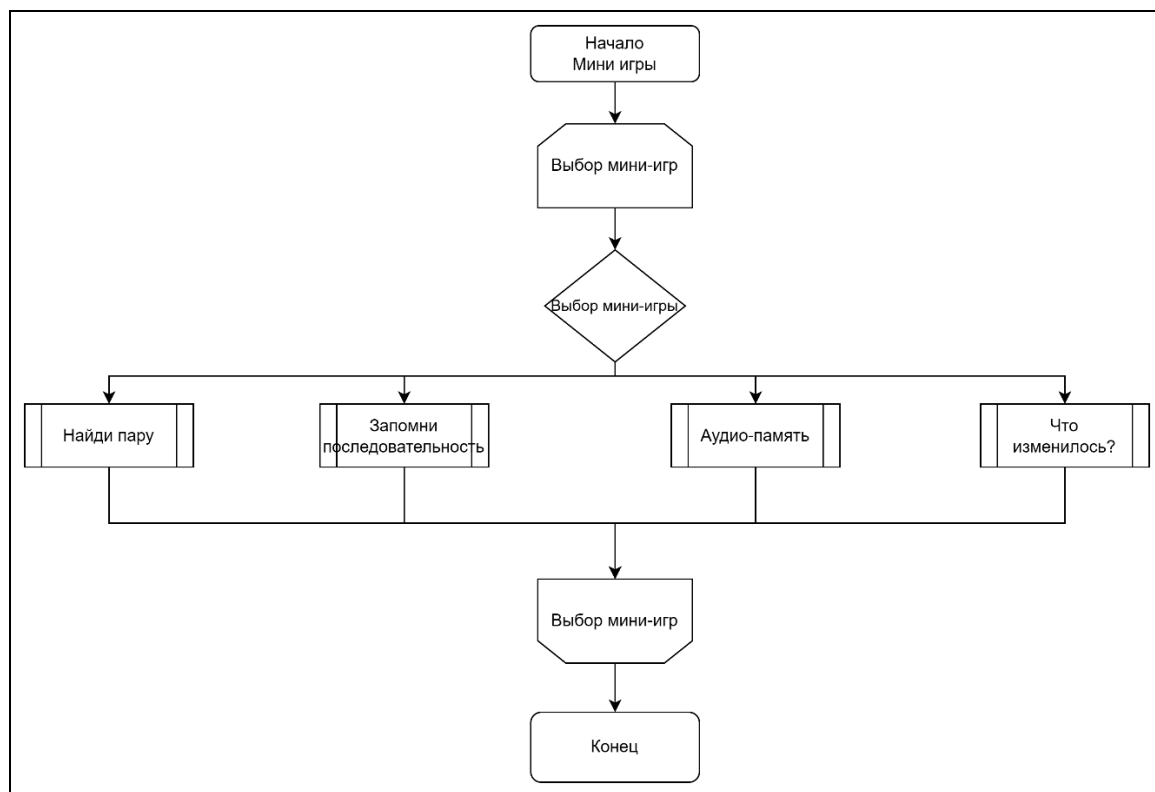


Рисунок 4.2.4 – Алгоритм выбора мини-игры

При выборе режима «Найди пару» пользователю предстает выбор между сложностями, которые влияют на размер сетки и количество находимых пар. После этого происходит запуск игры и продолжается до тех пор, пока все пары не будут найдены. В конце пользователю выведется окно с результатом прохождения и произойдет возвращение в главное меню. Алгоритм режима «Найди пару» представлен на рисунке 4.2.5.

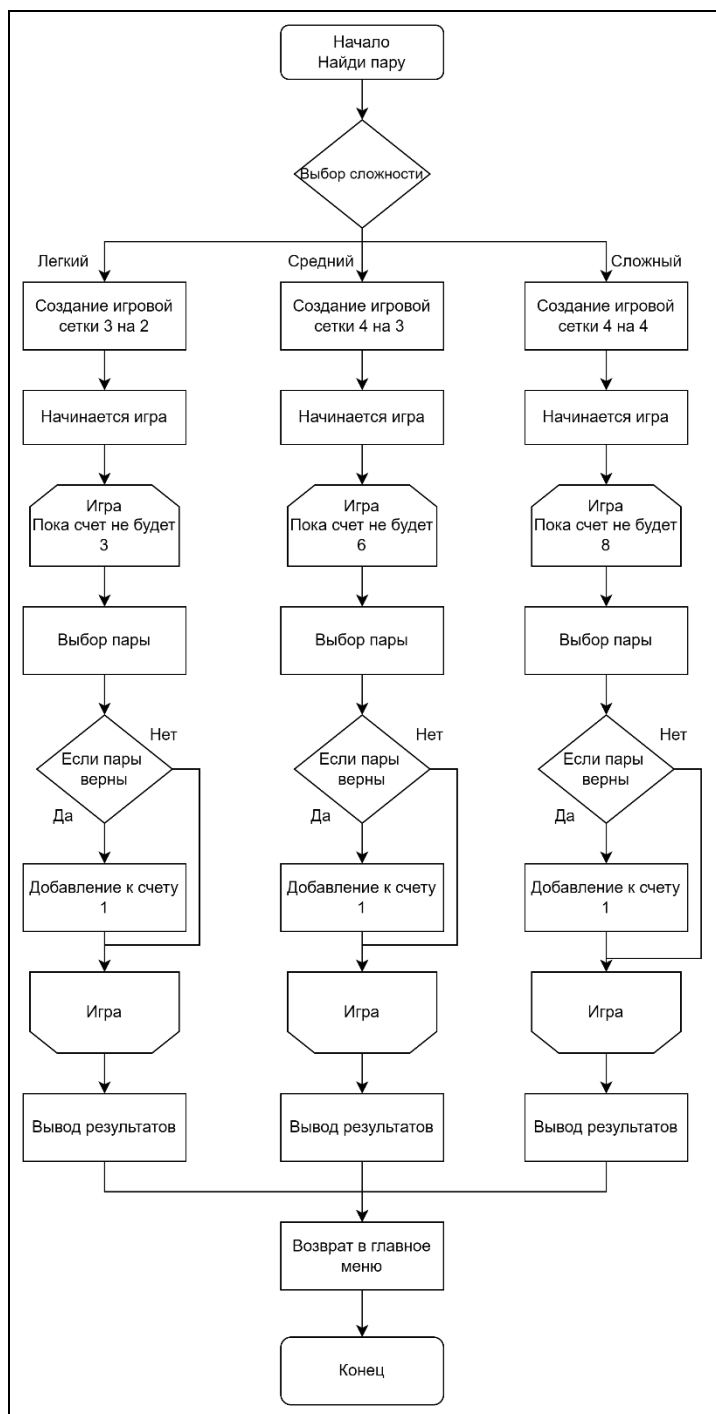


Рисунок 4.2.5 – Алгоритм режима «Найди пару»

При выборе режима «Запомни последовательность» пользователю предстает выбор между сложностями, которые влияет на время для запоминания и количество карточек. После этого происходит запуск игры и если, игрок выбрал верную карточку, то к следующей последовательности добавляется ещё одна. Игра продолжается до тех пор, пока не будет сделана ошибка. В итоге пользователю выведется окно с результатом прохождения и

произойдет возвращение в главное меню. Алгоритм режима «Запомни последовательность» представлен на рисунке 4.2.6.

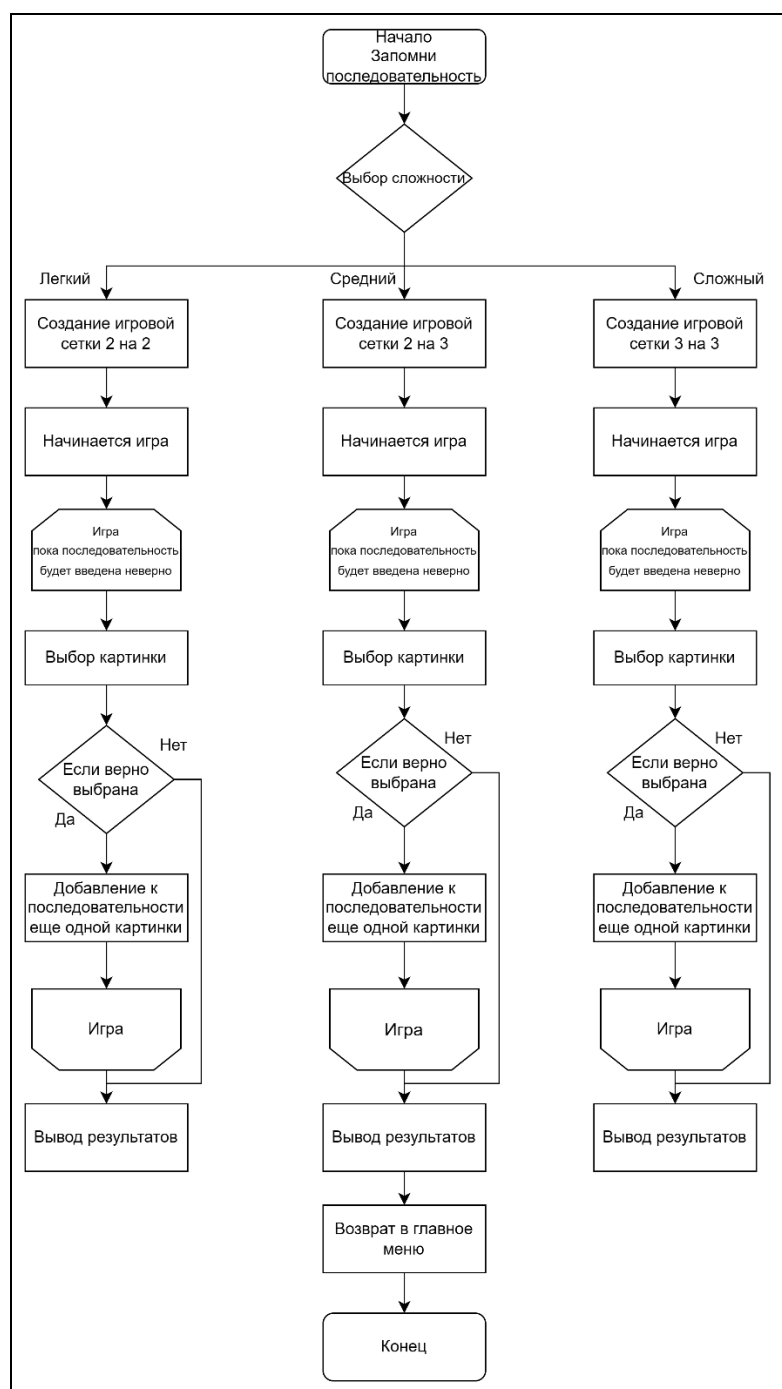


Рисунок 4.2.6 – Алгоритм режима «Завершение игры»

При выборе режима «Аудио-память» пользователю предстает выбор между сложностями, которые влияет на количество звуков. После этого происходит запуск игры и если игрок выбрал верный звук, то к следующему

ходу будет добавлен еще один. Игра продолжается до тех пор, пока не будет сделана ошибка. В итоге пользователю выведется окно с результатом прохождения и произойдет возвращение в главное меню. Алгоритм режима «Аудио-память» представлен на рисунке 4.2.7.

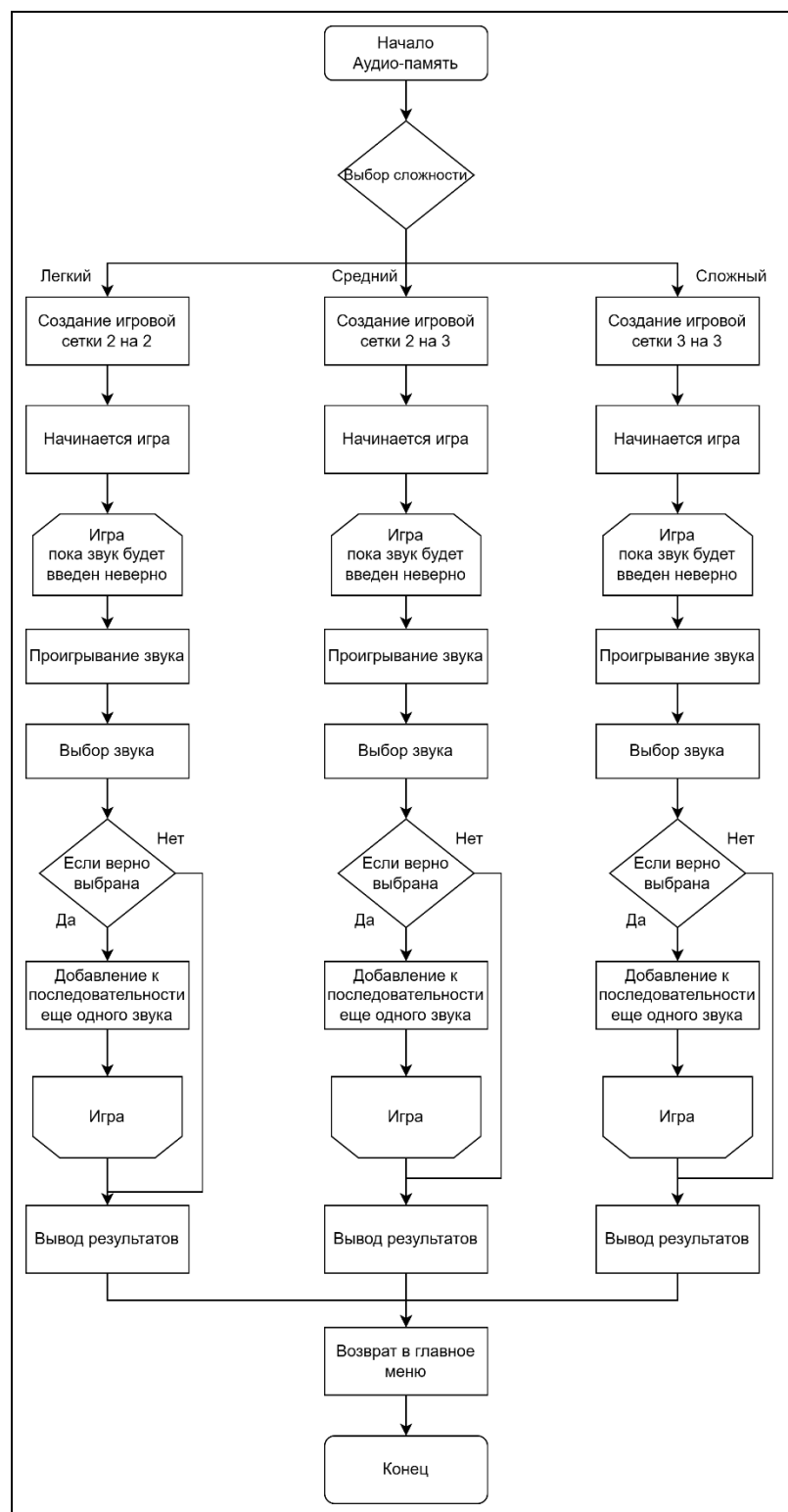


Рисунок 4.2.7 – Алгоритм режима «Аудио-память»

При выборе режима «Что изменилось?» пользователю предстает выбор между сложностями, которые влияют на время для запоминания и размер игровой сетки. После этого происходит запуск игры и если игрок выбрал верное изменение, то игроку добавятся очки и произойдет переход к следующему уровню. Игра продолжается до тех пор, пока не будет сделана ошибка. В итоге пользователю выведется окно с результатом прохождения и произойдет возвращение в главное меню. Алгоритм режима «Что изменилось?» представлен на рисунке 4.2.8

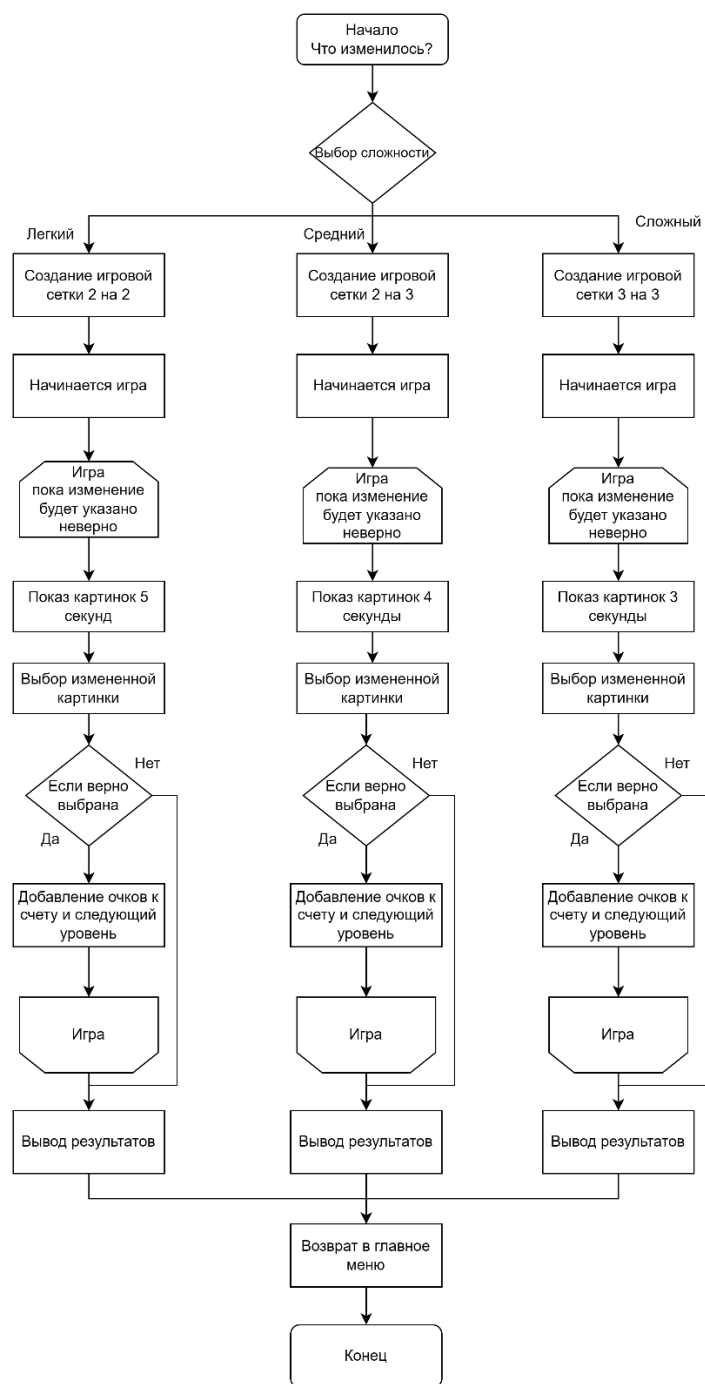


Рисунок 4.2.8 – Алгоритм режима «Что изменилось?»

4.2.3 Проектирование интерфейса

Приводится прототип интерфейса, нарисованный в Figma. Прототип главного меню представлен на рисунке 4.3.1.

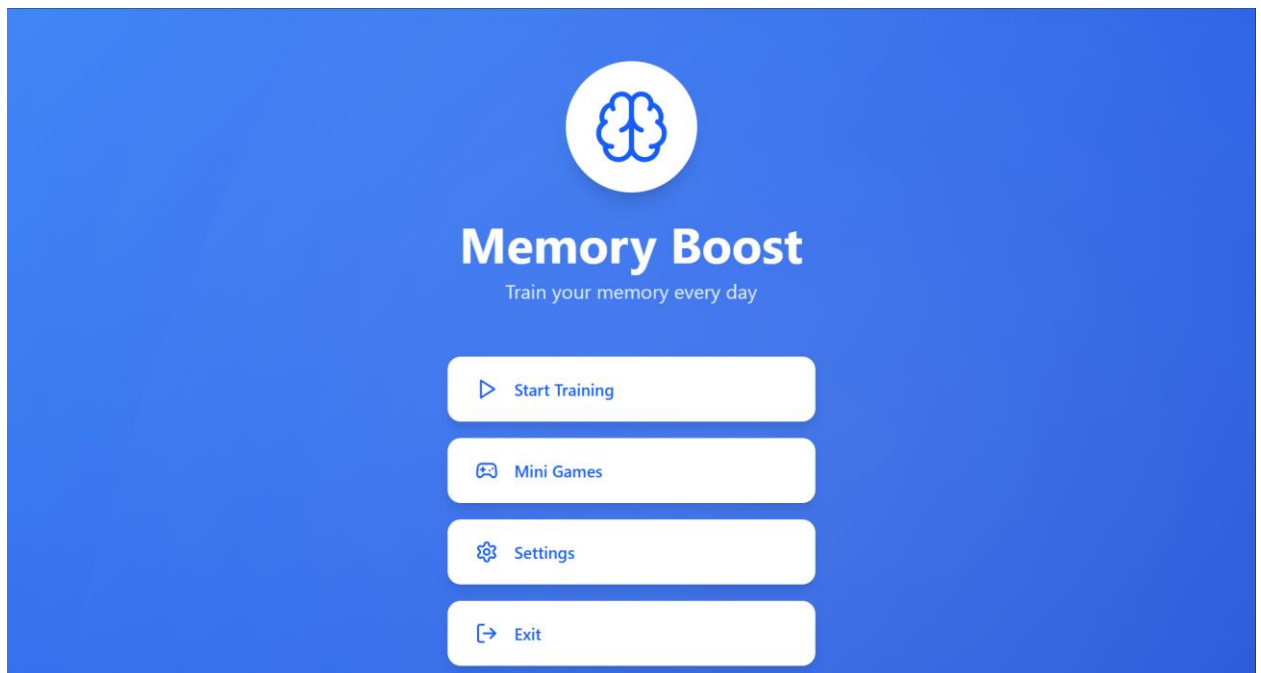


Рисунок 4.3.1 – Прототип главного меню приложения для развития памяти
Прототип меню настроек представлен на рисунке 4.3.2.

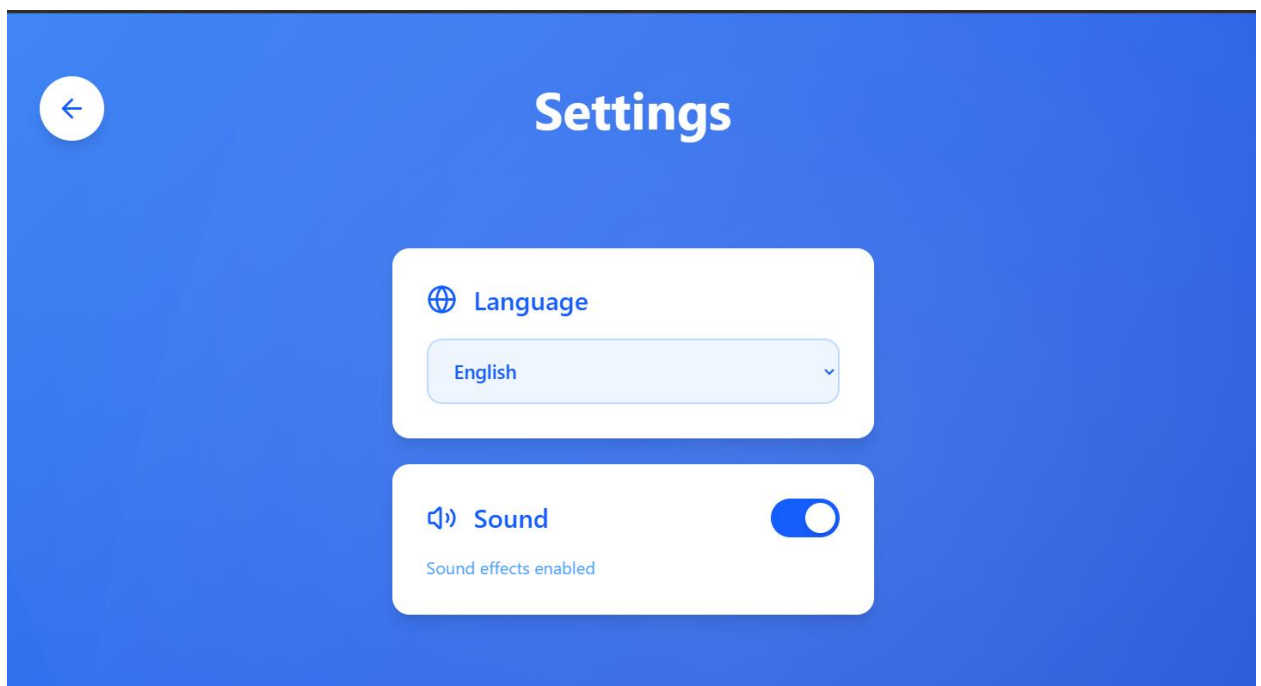


Рисунок 4.3.2 – Прототип меню настроек
Прототип меню выбора мини-игры представлен на рисунке 4.3.3.

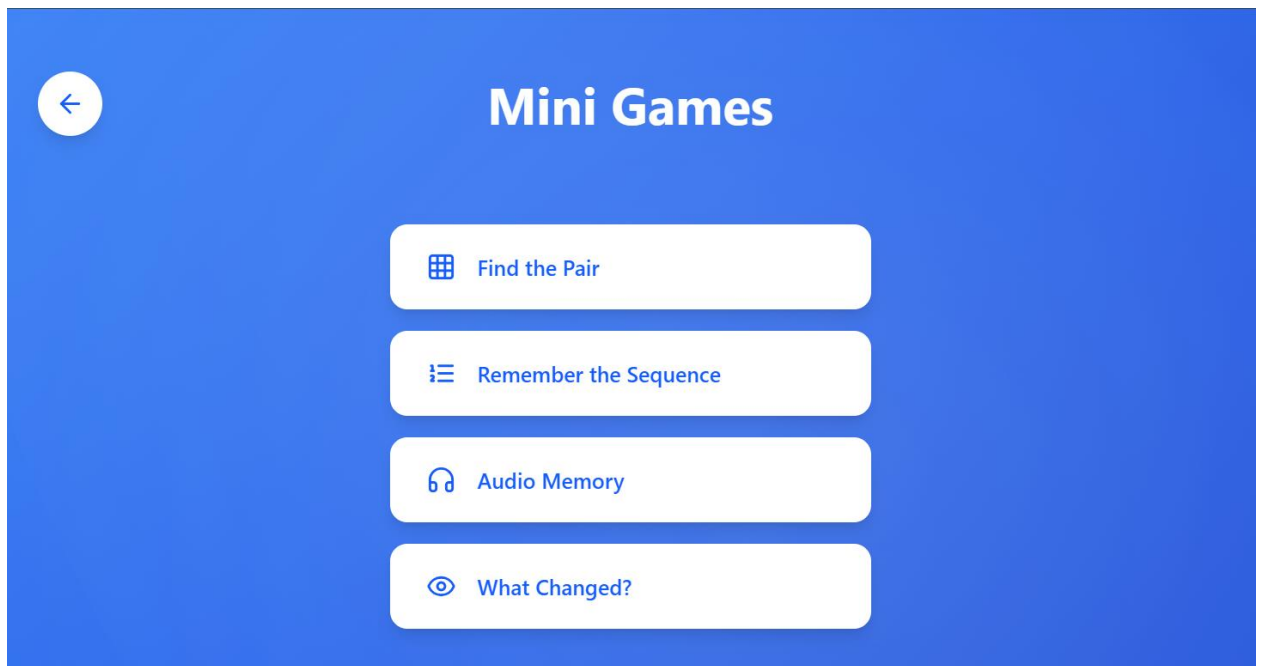


Рисунок 4.3.3 – Прототип меню выбора мини-игры

Прототип меню выбора уровня сложности показан на рисунке 4.3.4.

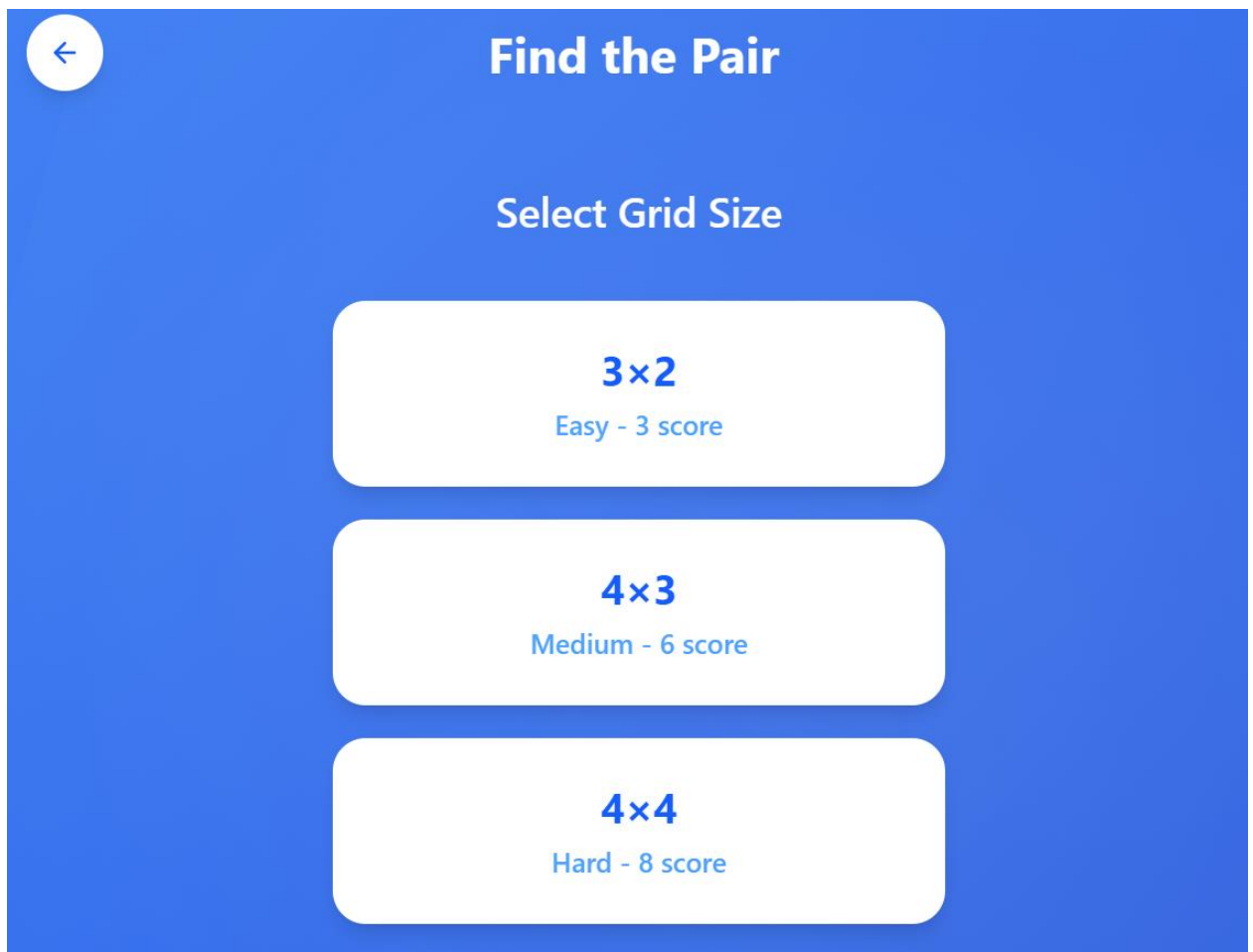


Рисунок 4.3.4 – Прототип меню выбора уровня сложности

Прототип интерфейса мини-игры указан на рисунке 4.3.5.

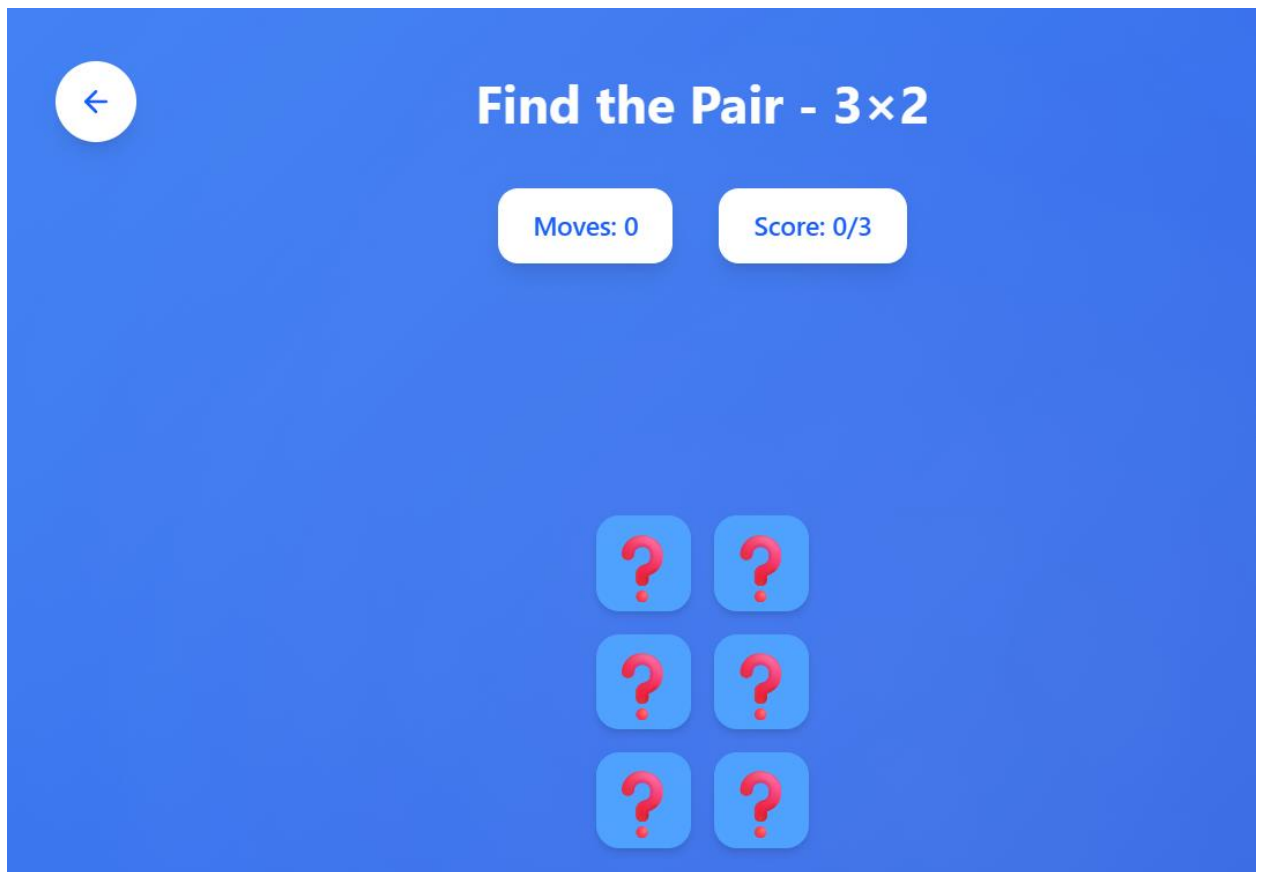


Рисунок 4.3.5 – Прототип интерфейса мини-игры

В данном разделе представлен прототип пользовательского интерфейса, включающий главное меню, экраны настроек, выбора мини-игры, уровня сложности и самой мини-игры. Разработанные схемы алгоритма и прототип наглядно отражают логику работы системы и взаимодействие пользователя с приложением.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования программы, разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано в среде PyCharm на языке Python с использованием библиотеки tkinter для создания графического интерфейса.

Программное обеспечение состоит из набора модулей (файлов), реализующих различные аспекты приложения. Главный модуль main.py создает экземпляр главного окна приложения App, который управляет отображением экранных форм. Каждая экранная форма (меню, настройки, конкретная мини-игра) реализована в виде отдельного класса.

Класс app является ядром всего приложения. Он создает главное окно, управляет переключением между экранами, хранит глобальные настройки и контролирует воспроизведение фоновой музыки. Структура класса представлена на рисунке 4.3.1.

```

import tkinter as tk
import winsound
import threading

from menus.main_menu import MainMenu
from menus.games_menu import GamesMenu
from menus.settings_menu import SettingsMenu
from menus.difficulty_menu import DifficultyMenu
from games.memory_pairs_game import MemoryPairsGame
from games.audio_memory_game import AudioMemoryGame
from games.remember_sequence_game import SequenceMemoryGame
from games.visual_memory_games import VisualMemoryGame
from training_mode import TrainingMode
from menus.stats_menu import StatsMenu

class App:

    def __init__(self, root):
        ...

    def play_music_loop(self):
        ...

    def show_frame(self, name):
        ...

if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()

```

Рисунок 4.3.1 – Класс App

Функция `__init__(self, root)` – конструктор класса. Инициализирует главное окно с заголовком `MemoryTrainer`, размером `1920×1080` и синим фоном. Создает контейнер для всех экранных форм. Устанавливает глобальные переменные: `language = "Русский"`, `sound_on = True`, `in_game = False` (флаг активности игры). В цикле создает все экранные формы (главное меню, меню игр, настройки, статистику, выбор сложности и мини-игры), сохраняя их в словарь `self.frames`. Запускает фоновую музыку.

Функция `play_music_loop(self)` – воспроизводит фоновую музыку, если звук включен и пользователь не находится в игре.

Использует `winsound.PlaySound` с флагами `SND_ASYNC` и `SND_LOOP` для асинхронного зацикленного воспроизведения файла `sounds/music.wav`.

Функция `show_frame(self, name)` – переключает экранные формы. Получает форму по имени из словаря `self.frames`, при необходимости обновляет её язык и поднимает на передний план методом `tkraise()`.

Класс `MainMenu` реализует главное меню приложения, отображаемое при запуске программы. Он содержит кнопки для навигации по всем разделам. Структура класса представлена на рисунке 4.3.2.

```
import tkinter as tk
from tkinter import Frame, Label, Button

class MainMenu(Frame):

    def __init__(self, parent, controller):
        ...
    def update_language(self):
        ...
```

Рисунок 4.3.2 – Класс `MainMenu`

Функция `__init__(self, parent, controller)` – создает центральную область с иконкой мозга, заголовком и кнопками: «Тренировка» (запускает режим тренировки), «Мини-игры» (переход к выбору игр), «Статистика», «Настройки» и «Выход» (закрывает приложение). Вызывает обновление языка.

Функция `update_language(self)` – обновляет тексты на кнопках и заголовках в зависимости от выбранного языка (русский или английский).

Класс `TrainingMode` реализует режим «Тренировка», в котором случайным образом выбираются три мини-игры с разными уровнями сложности, и после прохождения всех трех показывается итоговый результат. Структура класса показана на рисунке 4.3.3.

```

import random
import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import load, save

class TrainingMode:
    def __init__(self, controller):
        ...
    def start(self):
        ...
    def show_transition(self):
        ...
    def launch_game(self):
        ...
    def on_game_finished(self, score, level):
        ...
    def show_final_results(self):
        ...
    def go_to_menu(self):
        ...

```

Рисунок 4.3.3 – Класс TrainingMode

Функция `__init__(self, controller)` – инициализирует списки игр и уровней сложности, создает экран перехода (с информацией о предстоящей игре) и экран результатов (с итоговой статистикой).

Функция `start(self)` – запускает тренировку: случайным образом выбирает три игры и уровни сложности, сбрасывает счетчики и отображает экран перехода к первой игре.

Функция `show_transition(self)` – показывает информационный экран с названием игры, сложностью и номером текущего задания (1/3, 2/3, 3/3).

Функция `launch_game(self)` – запускает выбранную игру, передавая ей callback-функцию `on_game_finished` для получения результата.

Функция `on_game_finished(self, score, level)` – вызывается после завершения каждой мини-игры. Сохраняет результат, добавляет очки к общему счету и переходит к следующей игре.

Функция `show_final_results(self)` – отображает итоговые результаты тренировки: список всех трех игр с очками и уровнями, общую сумму очков и кнопку возврата в главное меню. Также обновляет файл статистики, увеличивая счетчик тренировок.

Функция `go_to_menu(self)` – возвращает пользователя в главное меню, запускает фоновую музыку.

Класс `StatsMenu` отображает статистику игр пользователя, загружая данные из файла `stats.json`. Структура класса представлена на рисунке 4.3.4.

```
import tkinter as tk
from tkinter import Frame, Label, Button
import json
import os

class StatsMenu(Frame):
    def __init__(self, parent, controller):
        ...
    def load_stats(self):
        ...
    def update_language(self):
        ...
```

Рисунок 4.3.4 – Класс `StatsMenu`

Функция `__init__(self, parent, controller)` – создает кнопку возврата, заголовок и области для отображения общей статистики и статистики по каждой игре.

Функция `load_stats(self)` – загружает данные из файла `stats.json` и возвращает их в виде словаря.

Функция `update_language(self)` – обновляет заголовок, загружает статистику и отображает: общее количество игр, количество тренировок, а также для каждой игры (по уровням сложности) лучшие результаты (очки, ходы или уровень).

Класс `SettingsMenu` предоставляет пользователю возможность изменять язык интерфейса и включать или отключать звуковое сопровождение. Структура класса указана на рисунке 4.3.5.

```
import tkinter as tk
from tkinter import Frame, Label, Button, IntVar

class SettingsMenu(Frame):

    def __init__(self, parent, controller):
        ...
    def update_language(self):
        ...
    def update_language_choice(self):
        ...
    def toggle_sound(self):
        ...
```

Рисунок 4.3.5 – Класс SettingsMenu

Функция `__init__(self, parent, controller)` – создает кнопку возврата, заголовок, радиокнопки выбора языка (русский/английский) и кнопку управления звуком.

Функция `update_language(self)` – обновляет тексты интерфейса в зависимости от выбранного языка и состояния звука.

Функция `update_language_choice(self)` – вызывается при смене языка, обновляет глобальную настройку и переводит все экранные формы.

Функция `toggle_sound(self)` – включает или выключает звук, останавливая или запуская фоновую музыку.

Класс `DifficultyMenu` (файл `menus/difficulty_menu.py`) предоставляет пользователю выбор уровня сложности перед началом игры. Структура класса показана на рисунке 4.3.6.


```

import tkinter as tk
from tkinter import Frame, Label, Button

class DifficultyMenu(Frame):
    def __init__(self, parent, controller):
        ...
    def set_game(self, game_name):
        ...
    def start_game(self, difficulty):
        ...
    def update_language(self):
        ...

```

Рисунок 4.3.6 – Класс DifficultyMenu

Функция `__init__(self, parent, controller)` – создает кнопку возврата, заголовок и три кнопки: «Легкий», «Средний», «Сложный».

Функция `set_game(self, game_name)` – запоминает, какая игра будет запущена после выбора сложности.

Функция `start_game(self, difficulty)` – запускает выбранную игру с указанным уровнем сложности.

Функция `update_language(self)` – обновляет тексты заголовка и кнопок.

Класс `GamesMenu` реализует экран выбора мини-игры в режиме «Свободная игра». Он содержит кнопки для перехода к каждой из четырех мини-игр. Структура класса представлена на рисунке 4.3.7.

```

import tkinter as tk
from tkinter import Frame, Label, Button

class GamesMenu(Frame):
    def __init__(self, parent, controller):
        ...
    def open_difficulty(self, game_name):
        ...
    def update_language(self):
        ...

```

Рисунок 4.3.7 – Класс GamesMenu

Функция `__init__(self, parent, controller)` – создает кнопку возврата в главное меню с помощью `Button`, заголовок экрана с помощью `Label`, а также четыре игровые кнопки: «Найди пару», «Аудио-память», «Запомни последовательность» и «Что изменилось?». Каждая кнопка при нажатии вызывает функцию `open_difficulty()` с соответствующим параметром (`pairs`, `audio`, `sequence`, `visual`). В конце вызывается функция `update_language()` для установки текста на кнопках.

Функция `open_difficulty(self, game_name)` – получает из контроллера экран выбора сложности `DifficultyMenu`, вызывает у него функцию `set_game()` для запоминания выбранной игры, затем переключается на экран выбора сложности с помощью `controller.show_frame()`.

Функция `update_language(self)` – обновляет тексты на всех элементах экрана в зависимости от выбранного языка. При русском языке заголовок становится «Мини-игры», кнопки получают названия «Найди пару», «Аудио-память», «Запомни последовательность», «Что изменилось?». При английском языке заголовок – «Mini Games», кнопки – «Find the Pair», «Audio Memory», «Remember the Sequence», «What Changed?».

Класс `VisualMemoryGame` – развивает зрительную память: пользователь запоминает расположение чисел на шести карточках, затем две карточки меняются местами, и нужно найти изменившиеся. Структура класса указана на рисунке 4.3.8.

```

import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random
import winsound

class VisualMemoryGame(Frame):
    def __init__(self, parent, controller):
        ...
    def update_language(self):
        ...
    def get_difficulty_text(self):
        ...
    def start_game(self, difficulty):
        ...
    def new_round(self):
        ...
    def update_timer(self):
        ...
    def change_cards(self):
        ...
    def fade_step(self, step):
        ...
    def swap_cards(self):
        ...
    def press(self, idx):
        ...
    def calculate_score(self):
        ...
    def end_game(self):
        ...
    def update_stats(self):
        ...
    def show_overlay(self, resume=False):
        ...

```

Рисунок 4.3.8 – Класс VisualMemoryGame

Функция `__init__(self, parent, controller)` – создает игровое поле, панель состояния (уровень, очки, таймер), кнопку паузы и оверлей для меню.

Функция `start_game(self, difficulty)` – запускает игру, устанавливая время запоминания (5/4/3 секунды в зависимости от сложности) и создавая сетку 2×3 с числами от 1 до 6.

Функция `new_round(self)` – начинает новый раунд: перемешивает числа, случайно выбирает две карточки для обмена и запускает таймер.

Функция `swap_cards(self)` – меняет местами выбранные карточки после завершения времени запоминания и разблокирует кнопки.

Функция `press(self, idx)` – обрабатывает нажатие: если пользователь выбрал обе изменившиеся карточки – переход к следующему уровню, если ошибся – завершение игры.

Функция `end_game(self)` – вычисляет очки, сохраняет статистику, показывает результат (в режиме тренировки вызывает `callback`).

Класс `SequenceMemoryGame` – тренирует память на последовательности: нужно запомнить и повторить порядок подсветки кнопок. Структура класса представлена на рисунке 4.3.9.

```

import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random

class SequenceMemoryGame(Frame):
    def __init__(self, parent, controller):
        ...
    def update_language(self):
        ...
    def get_difficulty_text(self):
        ...
    def start_game(self, difficulty):
        ...
    def add(self):
        ...
    def play(self):
        ...
    def flash(self, idx):
        ...
    def press(self, idx):
        ...
    def calculate_score(self):
        ...
    def end_game(self):
        ...
    def update_stats(self):
        ...
    def show_overlay(self, resume=False):
        ...
    def go_to_main_menu(self):
        ...
    def hide_overlay(self):
        ...
    def pause(self):
        ...

```

Рисунок 4.3.9 – Класс SequenceMemoryGame

Функция `start_game(self, difficulty)` – создает игровое поле размером 2×2 , 2×3 или 3×3 в зависимости от сложности.

Функция `add(self)` – добавляет случайную кнопку в последовательность.

Функция `play(self)` – воспроизводит последовательность, подсвечивая кнопки с интервалом 600 миллисекунд.

Функция `press(self, idx)` – проверяет правильность нажатия. При успешном повторении последовательность удлиняется, при ошибке игра завершается.

Класс `MemoryPairsGame` – классическая игра на поиск пар одинаковых карточек. Структура класса указана на рисунке 4.3.10.

```

import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random

class MemoryPairsGame(Frame):
    def __init__(self, parent, controller):
        ...
    def update_language(self):
        ...
    def get_difficulty_text(self):
        ...
    def start_game(self, difficulty):
        ...
    def flip(self, index):
        ...
    def check(self):
        ...
    def calculate_score(self):
        ...
    def end_game(self):
        ...
    def update_moves(self):
        ...
    def update_points(self):
        ...
    def show_end(self):
        ...
    def show_pause_menu(self):
        ...
    def show_overlay(self, title, resume=False):
        ...
    def hide_overlay(self):
        ...
    def go_to_main_menu(self):
        ...

```

Рисунок 4.3.10 – Класс MemoryPairsGame

Функция `start_game(self, difficulty)` – создает игровое поле: для легкого уровня 3×2 (6 карточек), для среднего 4×4 (16 карточек), для сложного 5×6 (30 карточек). Карточки перемешиваются.

Функция `flip(self, index)` – открывает карточку, показывает число и меняет цвет на голубой.

Функция `check(self)` – сравнивает две открытые карточки. При совпадении карточки становятся зелеными и отключаются, при несовпадении – переворачиваются обратно.

Функция `end_game(self)` – вычисляет очки (количество найденных пар минус лишние ходы), сохраняет статистику и показывает результат.

Класс `AudioMemoryGame` – развивает слуховую память: нужно запомнить и повторить последовательность звуков. Структура класса представлена на рисунке 4.3.11.


```

import tkinter as tk
from tkinter import Frame, Label, Button
import random
from functools import partial
from stats_manager import update
import winsound
import os

class AudioMemoryGame(Frame):
    def __init__(self, parent, controller):
        ...

    def update_language(self):
        ...

    def get_difficulty_text(self):
        ...

    def play_sound(self, idx):
        ...

    def start_game(self, difficulty):
        ...

    def add_to_sequence(self):
        ...

    def play_sequence(self):
        ...

    def unlock(self):
        ...

    def button_pressed(self, idx):
        ...

    def calculate_score(self):
        ...

    def end_game(self):
        ...

    def update_moves(self):
        ...

    def update_points(self):
        ...

    def show_pause(self):
        ...

    def show_fail_screen(self):
        ...

    def hide_overlay(self):
        ...

    def go_to_main_menu(self):
        ...

```

Рисунок 4.3.11 – Класс AudioMemoryGame

Функция `play_sound(self, idx)` – воспроизводит звуковой файл из папки `sounds/` (например, `sounds/ru/1.wav`).

Функция `start_game(self, difficulty)` – создает сетку кнопок размером 2×2 , 3×2 или 3×3 .

Функция `add_to_sequence(self)` – добавляет случайный звук в последовательность.

Функция `play_sequence(self)` – воспроизводит последовательность звуков с интервалом 800 миллисекунд.

Функция `button_pressed(self, idx)` – проверяет правильность нажатия. При ошибке игра завершается, при успехе последовательность удлиняется.

Класс `stats_manager` – содержит функцию для сохранения статистики в файл `stats.json`. Структура класса представлена на рисунке 4.3.12.

```
import json
import os
FILE = "stats.json"
def load():
    ...
def normalize_difficulty(diff, language):
    ...
def save(data):
    ...
def update(game, difficulty, win, score, level, moves, language):
    ...
def load_settings():
    ...
def save_settings(language, sound_on):
    ...
```

Рисунок 4.3.12 – Класс `stats_manager`

Функция `update(game, difficulty, win, score, level, moves, language)` – загружает текущую статистику, увеличивает счетчик сыгранных игр, обновляет лучшие результаты для указанной игры и уровня сложности (сохраняет лучшие очки, лучший уровень или лучшее количество ходов). При необходимости преобразует название сложности из английского в русский для единообразия хранения. Сохраняет обновленные данные обратно в файл.

`def load_settings()` – загружает настройки из файла `settings.json`.

`save_settings(language, sound_on)` – сохраняет переданные настройки (язык и состояние звука) в файл `settings.json` в формате JSON.

Так же на рисунке 4.3.13 представлена диаграмма классов.

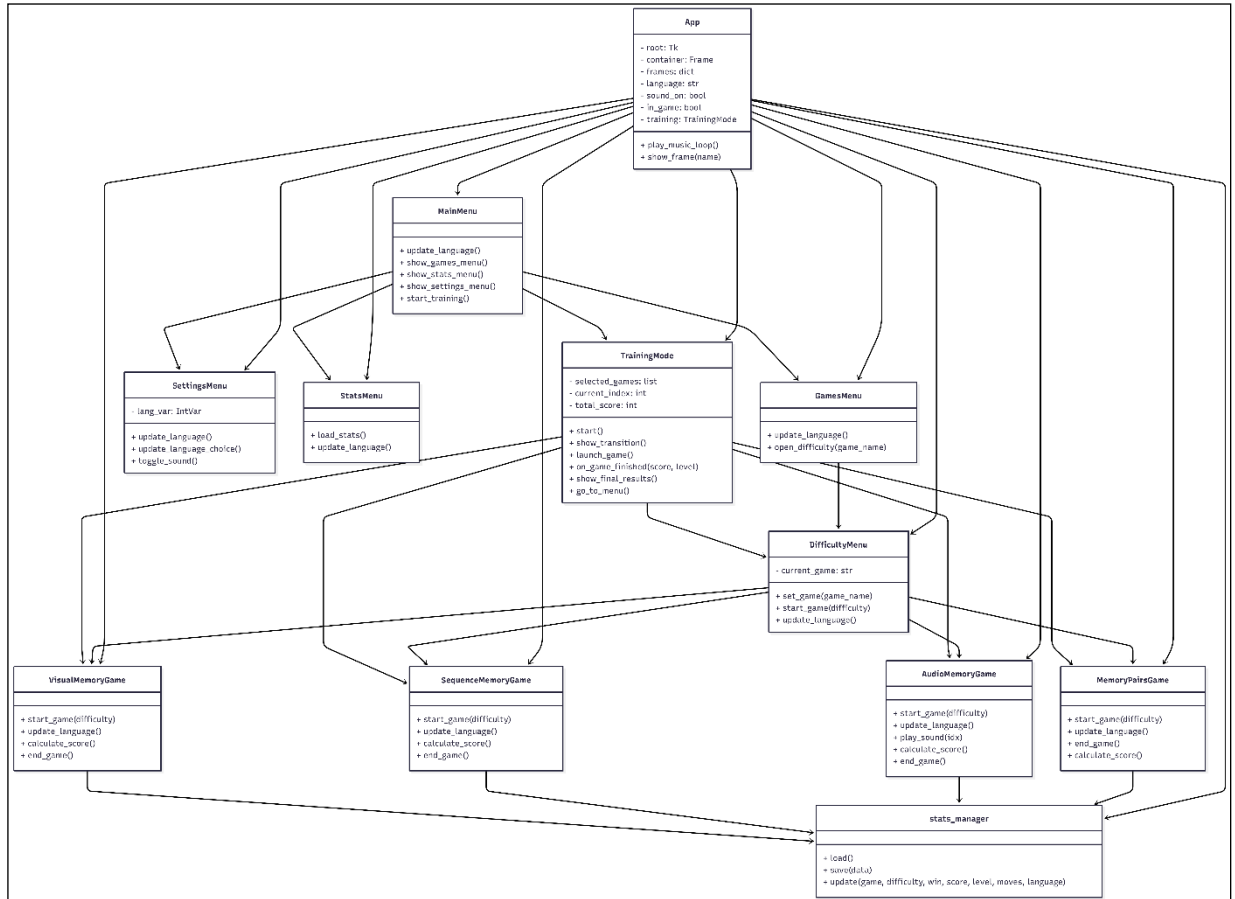


Рисунок 4.3.13 – Диаграмма классов

4.3.2 Разработка пользовательского интерфейса

Пользовательский интерфейс разработан с использованием библиотеки tkinter. Все экранные формы выполнены в едином стиле: синий фон (#0000FF), белый текст и крупные контрастные кнопки. Основные элементы управления: Button (кнопки для навигации и взаимодействия с игрой), Label (текстовые метки), Frame (контейнеры для группировки элементов).

Главное меню является стартовой формой приложения. Оно содержит кнопки для перехода в основные разделы. Внешний вид главного меню в русскоязычной версии представлен на рисунке 4.3.14.

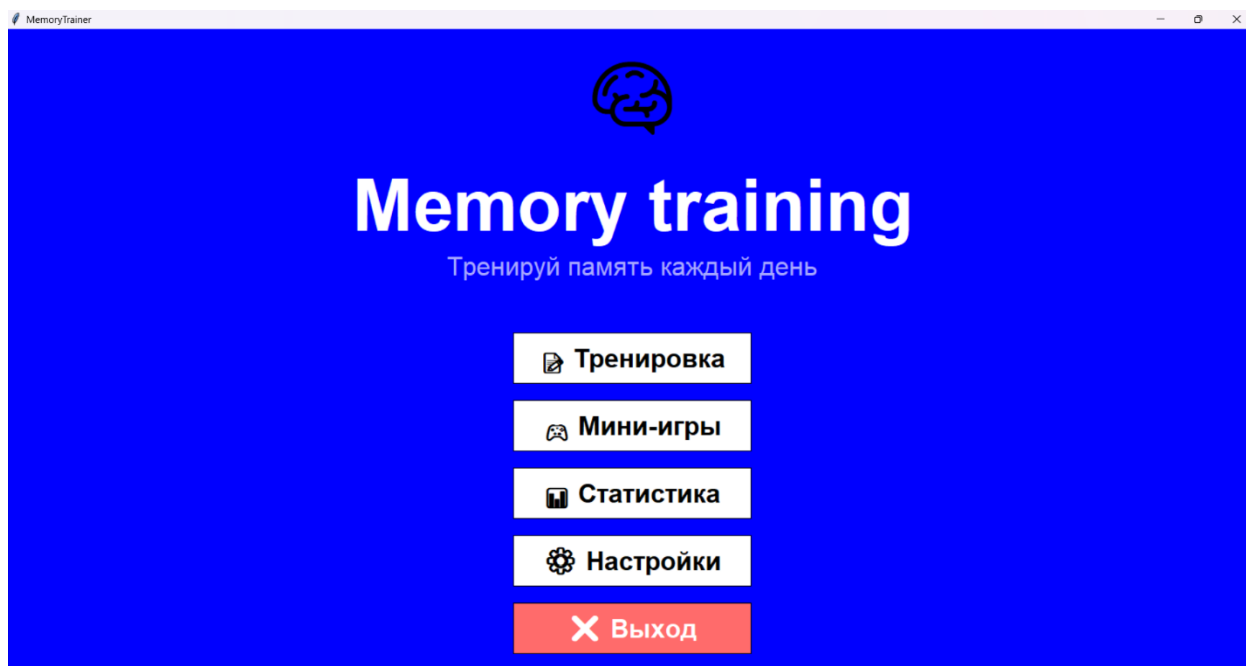


Рисунок 4.3.14 – Главное меню приложения

Экран выбора мини-игры предоставляет пользователю возможность выбрать одну из четырех мини-игр в режиме «Свободная игра». Для создания этого экрана используются кнопки, сгруппированные в центральном контейнере с помощью метода `pack()`. Реализация этого экрана показана на рисунке 4.3.15.

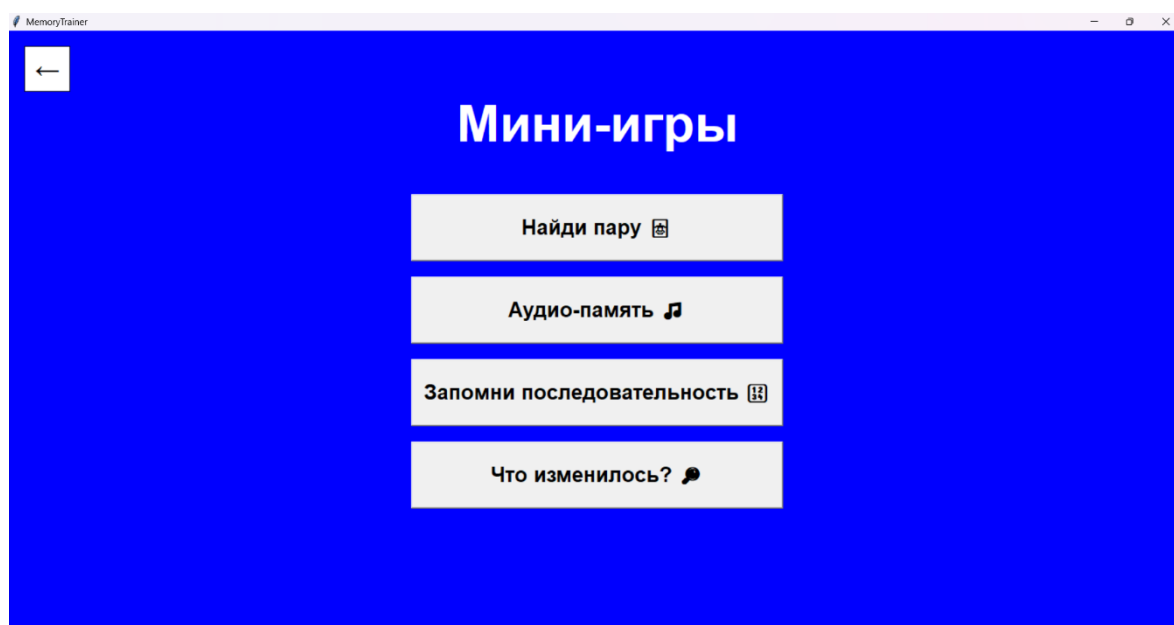


Рисунок 4.3.15 – Экран выбора мини-игры

Экран выбора сложности создается классом `DifficultyMenu`. Содержит три кнопки, созданные с помощью `Button` или `Radiobutton`: «Легкий»,

«Средний», «Сложный», которые вызывают функцию `start_game()` с соответствующим параметром. На экране также присутствует кнопка возврата в меню выбора игр. Внешний вид представлен на рисунке 4.3.16.

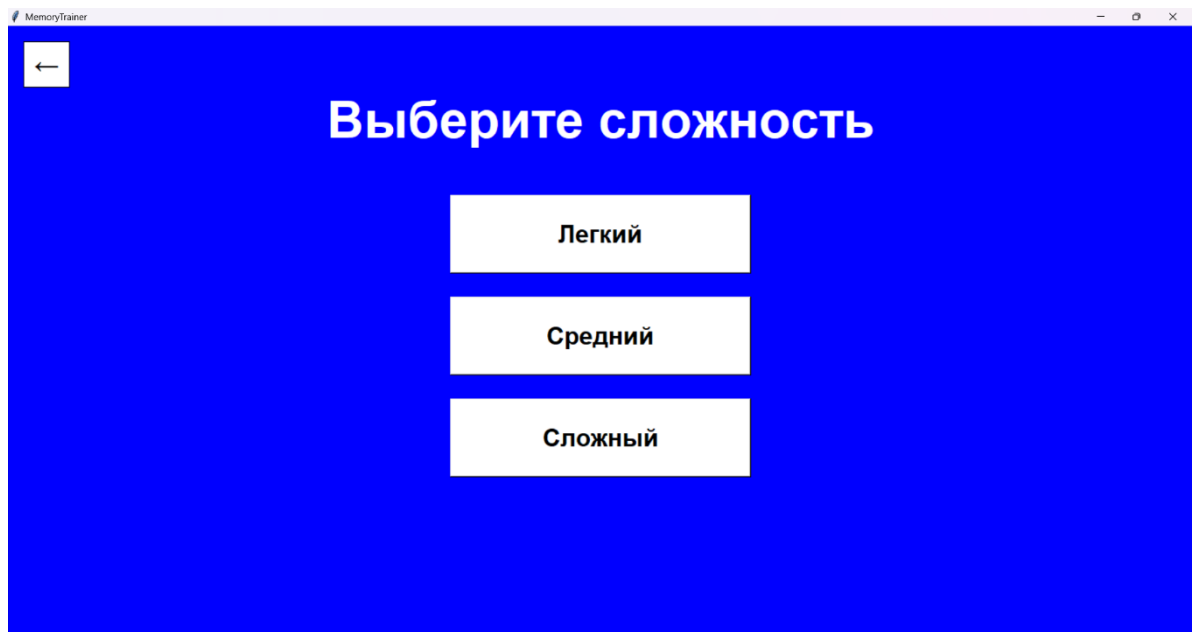


Рисунок 4.3.16 – Экран выбора сложности

Экран статистики создается классом `StatsMenu`. Функция `load_stats()` загружает данные из файла `stats.json`. Функция `update_language()` с помощью цикла и `Label` отображает общее количество игр, количество тренировок и лучшие результаты по каждой игре в разрезе уровней сложности. Внешний вид экрана статистика представлен на рисунке 4.3.17.

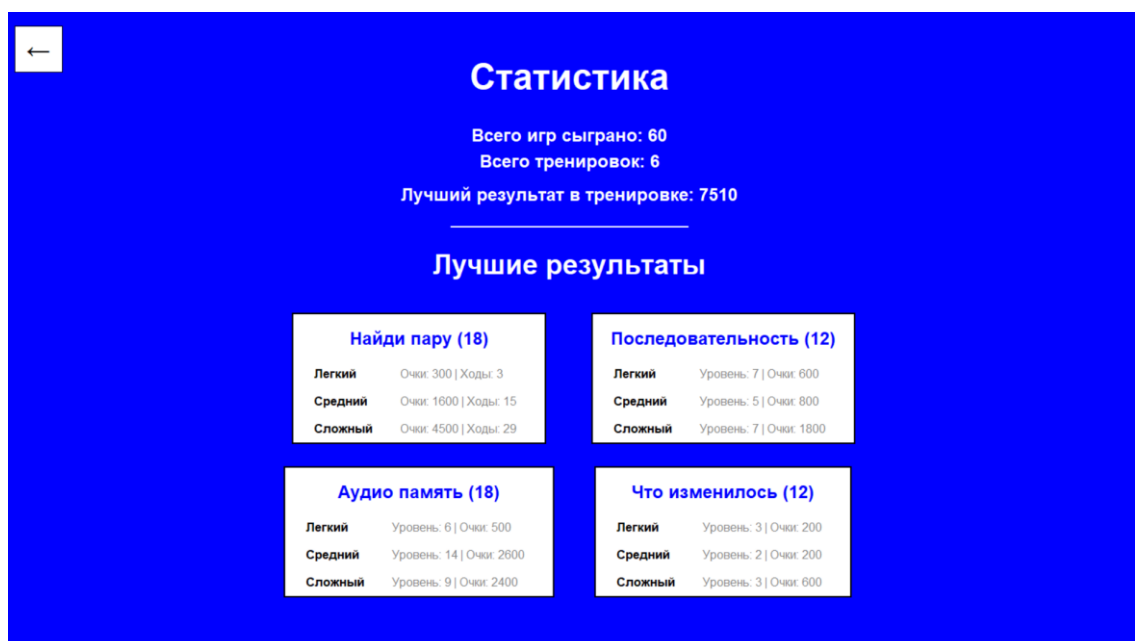


Рисунок 4.3.17 – Экран статистики

Экран настроек создается классом SettingsMenu. С помощью Radiobutton создаются две радиокнопки для выбора языка (русский/английский), связанные с переменной lang_var. Кнопка Button с функцией toggle_sound() управляет звуком. При смене языка функция update_language_choice() обновляет все экраны приложения. Внешний вид настроек представлен на рисунке 4.3.18.

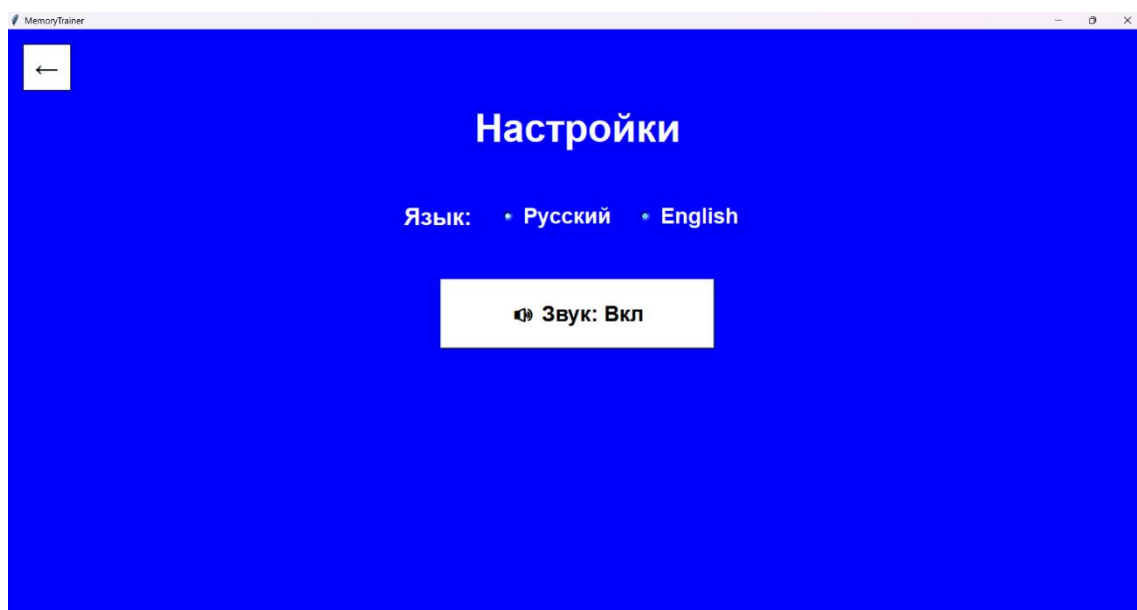


Рисунок 4.3.18 – Экран настроек

Экран перехода между играми в режиме тренировки создается классом TrainingMode. Функция show_transition() формирует информационное окно с помощью Label, которое отображает название предстоящей игры, уровень сложности и номер задания (1/3, 2/3, 3/3). Кнопка «Начать», созданная через Button, вызывает функцию launch_game() для запуска игры. Внешний вид экрана представлен на рисунке 4.3.19.

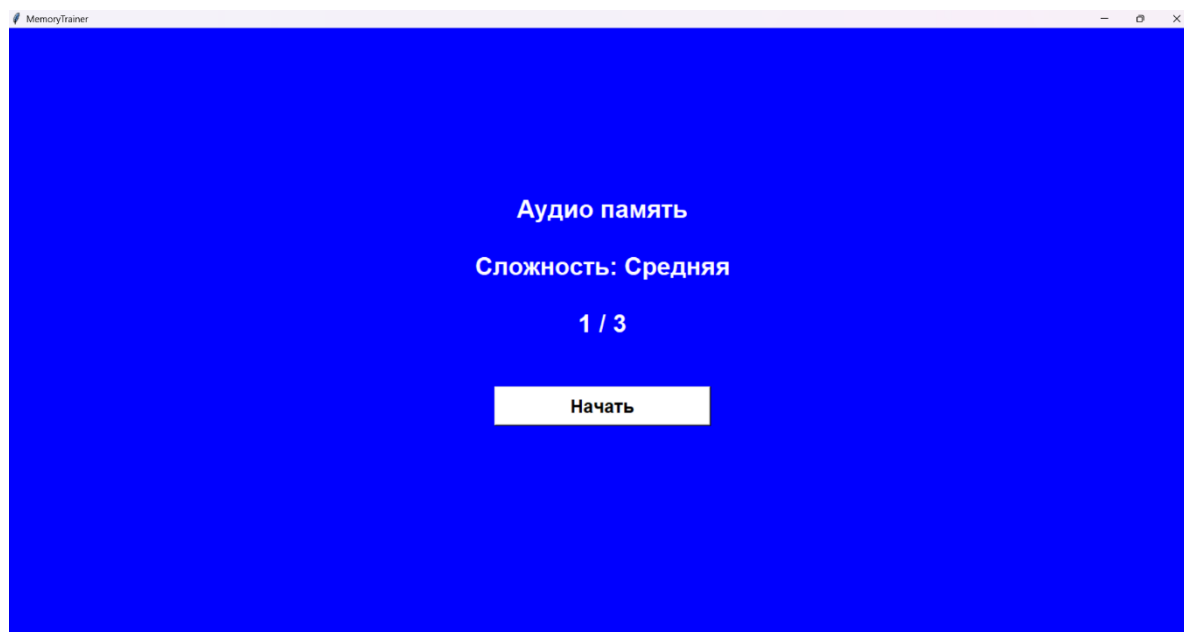


Рисунок 4.3.19 – Экран перехода между играми в режиме тренировки

Игровой экран «Найди пару» создается классом `MemoryPairsGame`. Функция `start_game()` с помощью цикла и метода `grid()` создает сетку из кнопок `Button`, размер которой зависит от сложности: 3×2 для легкого, 4×4 для среднего, 5×6 для сложного уровня. Панель состояния (ходы и очки) отображается с помощью `Label`. Кнопка паузы `Button` вызывает функцию `show_pause_menu()`. Внешний вид мини-игры указан на рисунке 4.3.20.



Рисунок 4.3.20 – Игровой экран мини-игры «Найди пару»

Игровой экран «Аудио-память» создается классом `AudioMemoryGame`. Функция `start_game()` создает сетку кнопок размером 2×2 , 3×2 или 3×3 . Функция `play_sound()` воспроизводит звуковые файлы, а `play_sequence()` запускает последовательное воспроизведение с помощью метода `after()`. Внешний вид мини-игры представлен на рисунке 4.3.21.

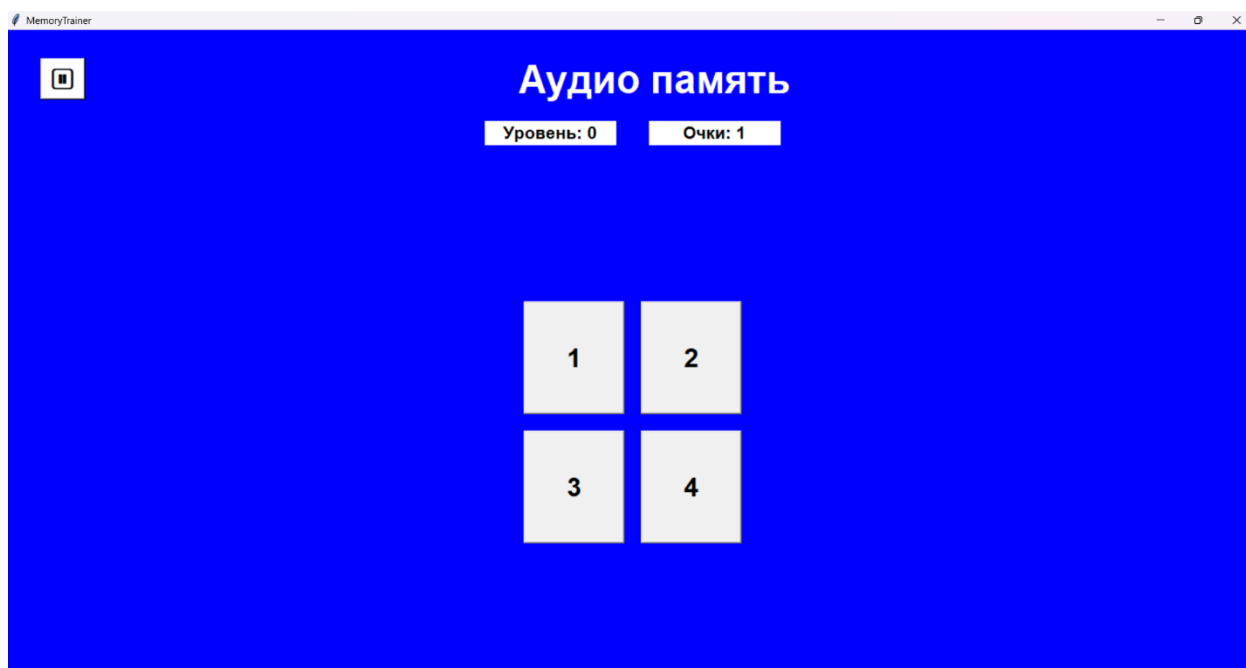


Рисунок 4.3.21 – Игровой экран мини-игры «Аудио-память»

Игровой экран «Запомни последовательность» создается классом `SequenceMemoryGame`. Функция `play()` с помощью метода `after()` и функции `flash()` последовательно подсвечивает кнопки с интервалом 600 миллисекунд. Внешний вид мини-игры представлен на рисунке 4.3.22.



Рисунок 4.3.22 – Игровой экран мини-игры «Запомни последовательность»

Игровой экран «Что изменилось?» создается классом VisualMemoryGame. Функция new_round() запускает таймер с помощью after() и функцию swap_cards() для обмена карточек местами. Внешний вид мини-игры указан на рисунке 4.3.23.

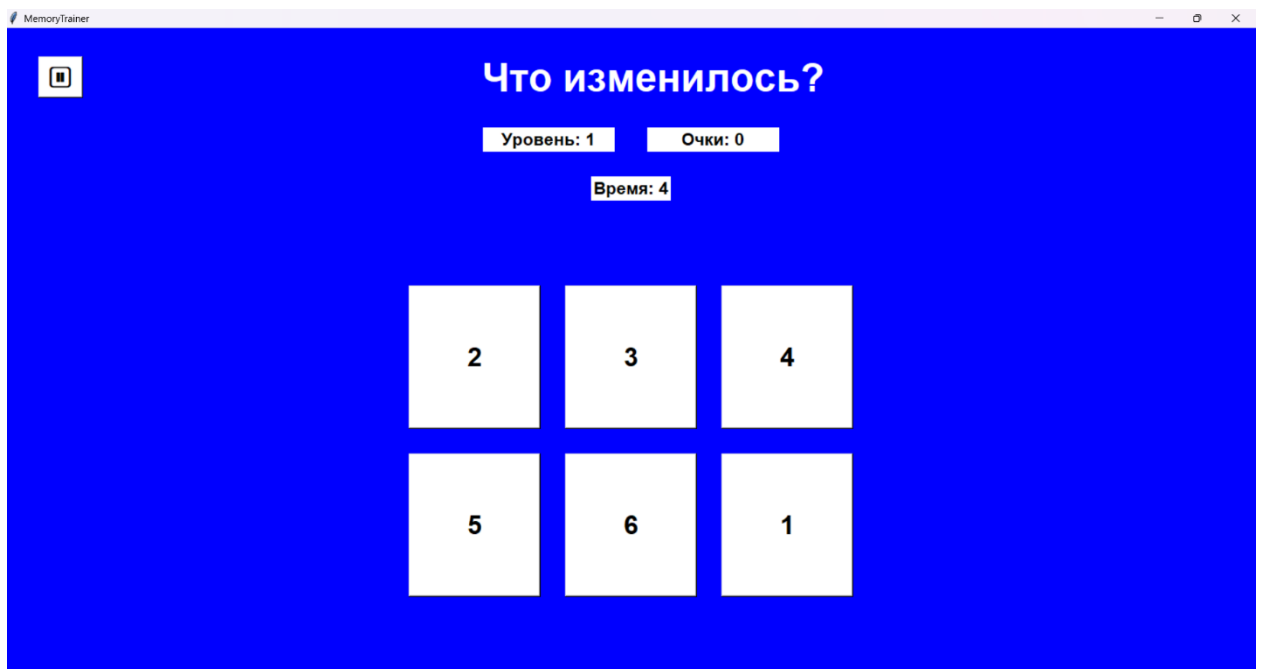


Рисунок 4.3.23 - Игровой экран мини-игры «Что изменилось?»

Экран паузы реализован с помощью оверлейного контейнера Frame, который накладывается на игровой экран методом place(). Функция

show_overlay() создает кнопки «Продолжить» (вызывает hide_overlay()) и «В главное меню» (вызывает go_to_main_menu()). Внешний вид экрана паузы представлен на рисунке 4.3.24.

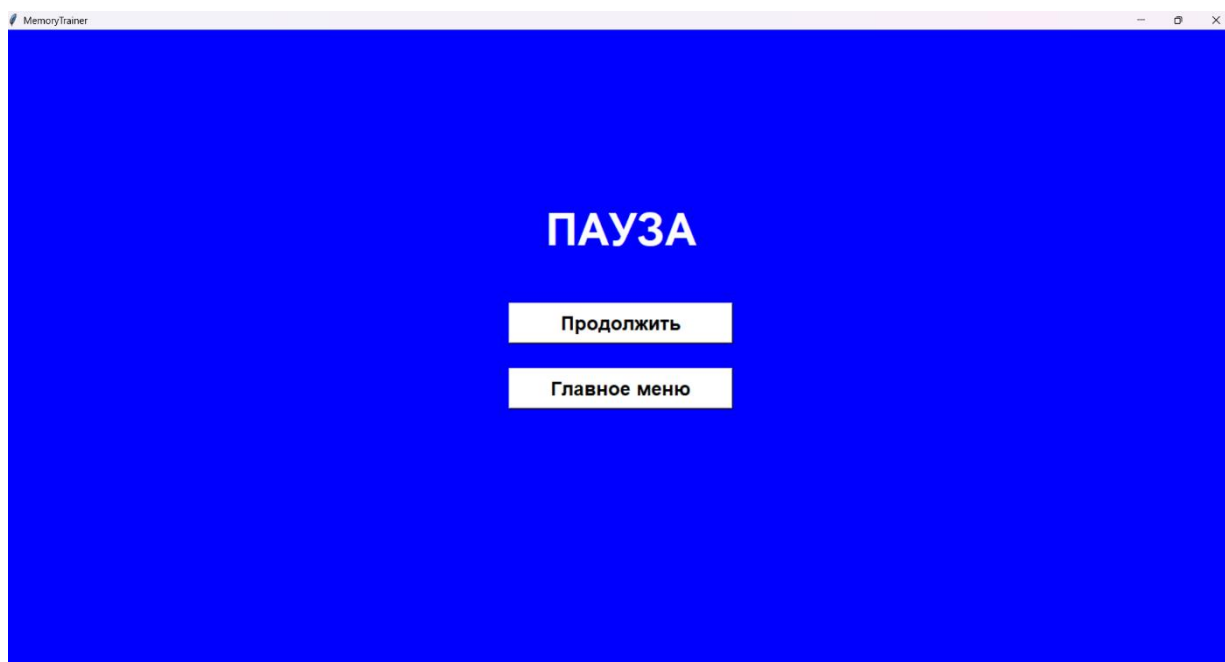


Рисунок 4.3.24 – Экран паузы

Экран результатов мини-игры создается классом каждой мини-игры. Функция show_overlay() или show_end() формирует окно с итоговыми очками и уровнем, а также кнопкой возврата в главное меню. Внешний вид экрана представлен на рисунке 4.3.25.



Рисунок 4.3.25 – Экран результатов мини-игры

Экран результатов тренировки создается классом `TrainingMode`. Функция `show_final_results()` в цикле создает текстовые метки `Label` для каждой из трех пройденных игр, отображая название, очки и уровень, а также итоговую сумму очков. Кнопка «В главное меню» вызывает функцию `go_to_menu()`. Экран представлен на рисунке 4.3.26.

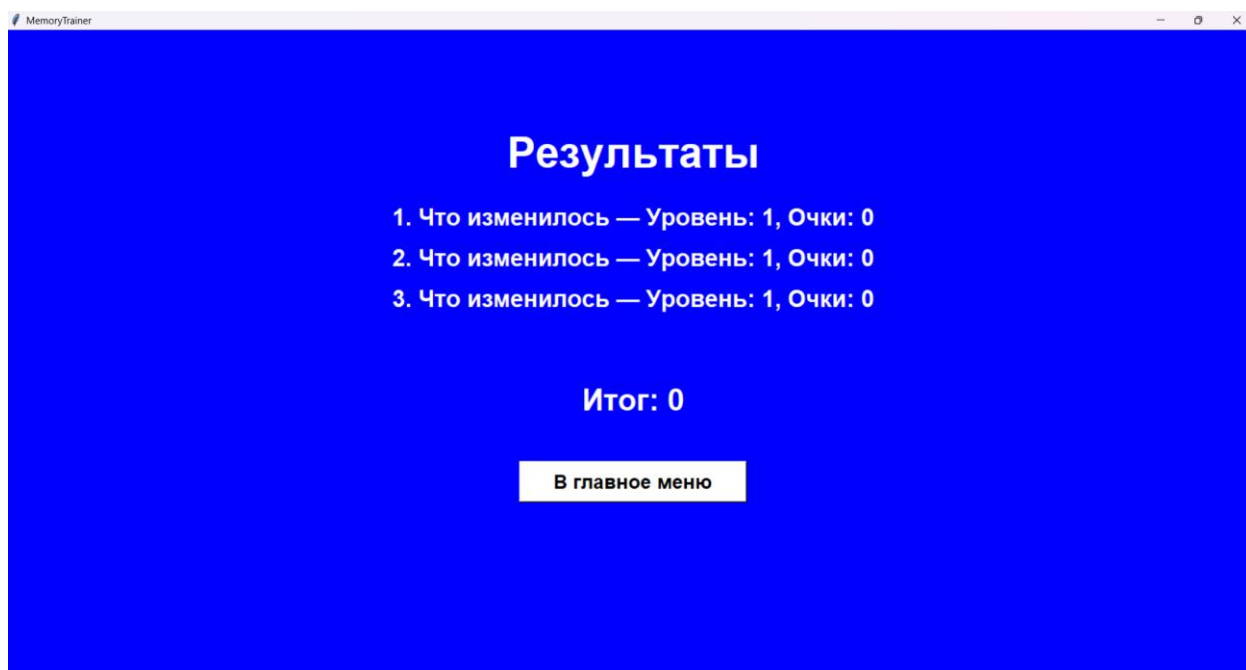


Рисунок 4.3.26 – Экран результатов тренировки

Вывод: в результате выполнения данного раздела были разработаны алгоритмы функционирования всех основных режимов и мини-игр приложения «Тренировка памяти». Реализован графический пользовательский интерфейс в соответствии с требованиями технического задания, обеспечивающий удобную навигацию, игровой процесс, а также функции отображения статистики и настройки приложения. Разработанное программное обеспечение готово к интеграции всех компонентов и проведению тестирования.

Приложение А

Исходный код

run.py

```
import tkinter as tk
import winsound
import threading

from menus.main_menu import MainMenu
from menus.games_menu import GamesMenu
from menus.settings_menu import SettingsMenu
from menus.difficulty_menu import DifficultyMenu
from games.memory_pairs_game import MemoryPairsGame
from games.audio_memory_game import AudioMemoryGame
from games.remember_sequence_game import SequenceMemoryGame
from games.visual_memory_games import VisualMemoryGame
from training_mode import TrainingMode
from menus.stats_menu import StatsMenu

class App:

    def __init__(self, root):
        self.root = root
        self.root.title("MemoryTrainer")

        self.normal_width = 1024
        self.normal_height = 768

        self.root.geometry(f"{self.normal_width}x{self.normal_height}")

        self.center_window()

        self.root.attributes('-fullscreen', True)

        self.root.bind('<Escape>', self.toggle_fullscreen)

        self.root.configure(bg="#0000FF")

        from stats_manager import load_settings

        settings = load_settings()

        self.language = settings.get("language", "Русский")
        self.sound_on = settings.get("sound_on", True)

        self.container = tk.Frame(root, bg="#0000FF")
        self.container.pack(fill="both", expand=True)

        self.frames = {}

        self.training = TrainingMode(self)

        for F in (MainMenu, GamesMenu, SettingsMenu, StatsMenu, DifficultyMenu,
                  MemoryPairsGame, AudioMemoryGame,
                  SequenceMemoryGame, VisualMemoryGame):
            frame = F(self.container, self)
            self.frames[F.__name__] = frame
            frame.place(relwidth=1, relheight=1)
```

```

        self.show_frame("MainMenu")

        self.in_game = False

        self.play_music_loop()

    def toggle_fullscreen(self, event=None):
        is_fullscreen = self.root.attributes('-fullscreen')

        if is_fullscreen:
            self.root.attributes('-fullscreen', False)
            self.root.geometry(f"{self.normal_width}x{self.normal_height}")
            self.center_window()
        else:
            self.normal_width = self.root.winfo_width()
            self.normal_height = self.root.winfo_height()
            self.root.attributes('-fullscreen', True)

    def center_window(self):
        self.root.update_idletasks()
        screen_width = self.root.winfo_screenwidth()
        screen_height = self.root.winfo_screenheight()
        x = (screen_width - self.normal_width) // 2
        y = (screen_height - self.normal_height) // 2
        self.root.geometry(f"{self.normal_width}x{self.normal_height}+{x}+{y}")

    def play_music_loop(self):
        if self.sound_on and not self.in_game:
            import winsound
            winsound.PlaySound(
                "sounds/music.wav",
                winsound.SND_FILENAME | winsound.SND_ASYNC | winsound.SND_LOOP
            )

    def show_frame(self, name):
        frame = self.frames[name]

        if hasattr(frame, "update_language"):
            frame.update_language()

        frame.tkraise()

if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()

```

main_menu.py

```

import tkinter as tk
from tkinter import Frame, Label, Button

class MainMenu(Frame):

    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

```

```

self.frame = Frame(self, bg="#0000FF")
self.frame.place(relx=0.5, rely=0.5, anchor="center")

self.brain_label = Label(self.frame, text="□", bg="#0000FF", font=("Arial", 80))
self.brain_label.pack(pady=(0, 20))

self.title_label = Label(self.frame, bg="#0000FF", fg="white", font=("Arial", 67,
"bold"))
self.title_label.pack()

self.subtitle_label = Label(self.frame, bg="#0000FF", fg="#AAAAFF",
font=("Arial", 24))
self.subtitle_label.pack(pady=(0, 50))

button_style = {
    "font": ("Arial", 24, "bold"),
    "width": 15,
    "height": 1,
    "bg": "white",
    "fg": "black",
    "bd": 1,
    "relief": "solid",
    "cursor": "hand2"
}

exit_style = button_style.copy()
exit_style.update({"bg": "#ff6b6b", "fg": "white"})

self.btn_train = Button(
    self.frame,
    command=lambda: self.controller.training.start(),
    **button_style
)
self.btn_train.pack(pady=10)

self.btn_games = Button(self.frame, command=lambda:
controller.show_frame("GamesMenu"), **button_style)
self.btn_games.pack(pady=10)

self.btn_stats = Button(
    self.frame,
    command=lambda: controller.show_frame("StatsMenu"),
    **button_style
)
self.btn_stats.pack(pady=10)

self.btn_settings = Button(self.frame, command=lambda:
controller.show_frame("SettingsMenu"), **button_style)
self.btn_settings.pack(pady=10)

self.btn_exit = Button(self.frame, command=self.controller.root.destroy,
**exit_style)
self.btn_exit.pack(pady=10)

self.update_language()

def update_language(self):
    if self.controller.language == "Русский":
        self.title_label.config(text="Memory training")

```

```

        self.subtitle_label.config(text="Тренируй память каждый день")
        self.btn_train.config(text="📝 Тренировка")
        self.btn_games.config(text="🎮 Мини-игры")
        self.btn_stats.config(text="📊 Статистика")
        self.btn_settings.config(text="⚙️ Настройки")
        self.btn_exit.config(text="✖ Выход")
    else:
        self.title_label.config(text="Memory training")
        self.subtitle_label.config(text="Train your memory every day")
        self.btn_train.config(text="📝 Training")
        self.btn_games.config(text="🎮 Mini-games")
        self.btn_stats.config(text="📊 Statistics")
        self.btn_settings.config(text="⚙️ Settings")
        self.btn_exit.config(text="✖ Exit")

```

training_mode.py

```

import random
import json
import tkinter as tk
from tkinter import Frame, Label, Button

class TrainingMode:
    def __init__(self, controller):
        self.controller = controller
        self.results = []

        self.games_pool = [
            "VisualMemoryGame",
            "AudioMemoryGame",
            "MemoryPairsGame",
            "SequenceMemoryGame"
        ]
        self.difficulties = ["easy", "medium", "hard"]

        self.selected_games = []
        self.current_index = 0
        self.total_score = 0

        # ===== НАЗВАНИЯ =====
        self.names = {
            "ru": {
                "VisualMemoryGame": "Что изменилось",
                "AudioMemoryGame": "Аудио память",
                "MemoryPairsGame": "Найди пару",
                "SequenceMemoryGame": "Запомни последовательность"
            },
            "en": {
                "VisualMemoryGame": "What Changed",
                "AudioMemoryGame": "Audio Memory",
                "MemoryPairsGame": "Find the Pair",
                "SequenceMemoryGame": "Remember Sequence"
            }
        }

        self.diff_names = {
            "ru": {"easy": "Лёгкая", "medium": "Средняя", "hard": "Сложная"},

```

```

        "en": {"easy": "Easy", "medium": "Medium", "hard": "Hard"}
    }

    # ===== ЭКРАН ПЕРЕХОДА =====
    self.transition_frame = Frame(controller.container, bg="#0000FF")

    box = Frame(self.transition_frame, bg="#0000FF", width=700, height=450)
    box.place(relx=0.5, rely=0.5, anchor="center")
    box.pack_propagate(False)

    self.transition_label = Label(
        box, font=("Arial", 24, "bold"),
        bg="#0000FF", fg="white",
        justify="center", wraplength=600
    )
    self.transition_label.pack(pady=40)

    Button(
        box, text="Начать", font=("Arial", 18, "bold"),
        bg="white", width=18,
        command=self.launch_game
    ).pack(pady=20)

    # ===== ЭКРАН РЕЗУЛЬТАТОВ =====
    self.result_frame = Frame(controller.container, bg="#0000FF")

    self.result_box = Frame(self.result_frame, bg="#0000FF", width=900, height=600)
    self.result_box.place(relx=0.5, rely=0.5, anchor="center")
    self.result_box.pack_propagate(False)

    # ===== СТАРТ =====
    def start(self):
        self.results.clear()
        self.selected_games = [
            (random.choice(self.games_pool), random.choice(self.difficulties))
            for _ in range(3)
        ]
        self.current_index = self.total_score = 0
        self.show_transition()

    # ===== ПЕРЕХОД =====
    def show_transition(self):
        if self.current_index >= 3:
            return self.show_final_results()

        lang = "ru" if self.controller.language == "Русский" else "en"

        game, diff = self.selected_games[self.current_index]

        text = (
            f"{self.names[lang][game]}\n\n"
            f"{'Сложность' if lang == 'ru' else 'Difficulty'}: "
            f"{self.diff_names[lang][diff]}\n\n"
            f"{self.current_index + 1} / 3"
        )

        self.transition_label.config(text=text)
        self.transition_frame.place(relwidth=1, relheight=1)
        self.transition_frame.lift()

```



```

# ===== ЗАПУСК =====
def launch_game(self):
    self.transition_frame.place_forget()

    game_name, difficulty = self.selected_games[self.current_index]
    game = self.controller.frames[game_name]

    game.training_mode = True
    game.training_callback = self.on_game_finished

    self.controller.show_frame(game_name)
    game.start_game(difficulty)

# ===== КОНЕЦ ИГРЫ =====
def on_game_finished(self, score, level):
    game, diff = self.selected_games[self.current_index]

    self.results.append((game, diff, score, level))
    self.total_score += score
    self.current_index += 1

    self.controller.root.after(700, self.show_transition)

# ===== ФИНАЛ =====
def show_final_results(self):
    with open("stats.json", "r", encoding="utf-8") as f:
        stats = json.load(f)

    if self.results:
        stats["training_runs"] = stats.get("training_runs", 0) + 1

        current_best = stats.get("best_training_score", 0)
        if self.total_score > current_best:
            stats["best_training_score"] = self.total_score

    with open("stats.json", "w", encoding="utf-8") as f:
        json.dump(stats, f, ensure_ascii=False, indent=4)

    for w in self.result_box.winfo_children():
        w.destroy()

    lang = "ru" if self.controller.language == "Русский" else "en"

    Label(
        self.result_box,
        text="Результаты" if lang == "ru" else "Results",
        font=("Arial", 40, "bold"),
        bg="#0000FF", fg="white"
    ).pack(pady=20)

    for i, (game, _, score, level) in enumerate(self.results, 1):
        name = self.names[lang][game]

        if game == "MemoryPairsGame":
            text = f"{i}. {name} – {'Очки' if lang == 'ru' else 'Score'}: {score},\n"
            text += f"{'Ходы' if lang == 'ru' else 'Moves'}: {level}"
        else:
            text = f"{i}. {name} – {'Уровень' if lang == 'ru' else 'Level'}: {level},\n"
            text += f"{'Очки' if lang == 'ru' else 'Score'}: {score}"

```

```

        Label(
            self.result_box, text=text,
            font=("Arial", 22, "bold"),
            bg="#0000FF", fg="white",
            wraplength=800, justify="center"
        ).pack(pady=5)

    Label(
        self.result_box,
        text=f"\n{'Итого' if lang == 'ru' else 'Total'}: {self.total_score}",
        font=("Arial", 28, "bold"),
        bg="#0000FF", fg="white"
    ).pack(pady=30)

    if stats.get("best_training_score", 0) == self.total_score and self.total_score >
0:
        Label(
            self.result_box,
            text="🏆 НОВЫЙ РЕКОРД! 🏆" if lang == "ru" else "🏆 NEW RECORD! 🏆",
            font=("Arial", 20, "bold"),
            bg="#0000FF", fg="yellow"
        ).pack(pady=(0, 20))

    Button(
        self.result_box,
        text="В главное меню" if lang == "ru" else "Main Menu",
        font=("Arial", 18, "bold"),
        bg="white", width=18,
        command=self.go_to_menu
    ).pack(pady=20)

    self.result_frame.place(relwidth=1, relheight=1)
    self.result_frame.lift()

# ===== В МЕНЮ =====
def go_to_menu(self):
    self.result_frame.place_forget()
    self.controller.in_game = False
    self.controller.play_music_loop()
    self.controller.show_frame("MainMenu")

```

games_menu.py

```

import tkinter as tk
from tkinter import Frame, Label, Button

class GamesMenu(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

    # Кнопка назад
    Button(
        self, text="←", font=("Arial", 24, "bold"),
        bg="white", fg="black", relief="solid", bd=1,
        cursor="hand2",
        command=lambda: controller.show_frame("MainMenu")
    ).place(x=20, y=20, width=60, height=60)

```

```

self.title = Label(self, bg="#0000FF", fg="white", font=("Arial", 50, "bold"))
self.title.pack(pady=(80, 40))

# Игры: (ключ, кнопка)
self.buttons = {
    "pairs": Button(self, font=("Arial", 20, "bold"), width=28, height=2,
        command=lambda: self.open_difficulty("pairs")),
    "audio": Button(self, font=("Arial", 20, "bold"), width=28, height=2,
        command=lambda: self.open_difficulty("audio")),
    "sequence": Button(self, font=("Arial", 20, "bold"), width=28, height=2,
        command=lambda: self.open_difficulty("sequence")),
    "visual": Button(self, font=("Arial", 20, "bold"), width=28, height=2,
        command=lambda: self.open_difficulty("visual")),
}

for btn in self.buttons.values():
    btn.pack(pady=10)

self.update_language()

def open_difficulty(self, game_name):
    menu = self.controller.frames["DifficultyMenu"]
    menu.set_game(game_name)
    self.controller.show_frame("DifficultyMenu")

def update_language(self):
    ru = self.controller.language == "Русский"

    self.title.config(text="Мини-игры" if ru else "Mini Games")

    texts = {
        "pairs": ("Найди пару ♠", "Find the Pair ♠"),
        "audio": ("Аудио-память 🎵", "Audio Memory 🎵"),
        "sequence": ("Запомни последовательность 📋", "Remember the Sequence 📋"),
        "visual": ("Что изменилось? 🔍", "What Changed? 🔍"),
    }

    for key, btn in self.buttons.items():
        btn.config(text=texts[key][0 if ru else 1])

```

settings_menu.py

```

import tkinter as tk
from tkinter import Frame, Label, Button, IntVar
from stats_manager import save_settings

class SettingsMenu(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

        # ===== STATE =====
        self.lang_var = IntVar(value=0 if controller.language == "Русский" else 1)

        # ===== BACK =====
        Button(
            self, text="←", font=("Arial", 24, "bold"),

```

```

        bg="white", fg="black", width=3,
        relief="solid", bd=1, cursor="hand2",
        command=lambda: controller.show_frame("MainMenu")
    ).place(x=20, y=20)

# ===== TITLE =====
self.title = Label(self, bg="#0000FF", fg="white",
                    font=("Arial", 40, "bold"))
self.title.pack(pady=(100, 40))

# ===== LANGUAGE =====
lang_frame = Frame(self, bg="#0000FF")
lang_frame.pack(pady=20)

self.lang_label = Label(lang_frame, bg="#0000FF",
                        fg="white", font=("Arial", 24, "bold"))
self.lang_label.pack(side="left", padx=(0, 20))

self.rb_ru = tk.Radiobutton(
    lang_frame, text="Русский",
    variable=self.lang_var, value=0,
    font=("Arial", 22, "bold"),
    bg="#0000FF", fg="white",
    selectcolor="#66CCFF",
    activebackground="#0000FF",
    command=self.change_language
)
self.rb_en = tk.Radiobutton(
    lang_frame, text="English",
    variable=self.lang_var, value=1,
    font=("Arial", 22, "bold"),
    bg="#0000FF", fg="white",
    selectcolor="#66CCFF",
    activebackground="#0000FF",
    command=self.change_language
)

self.rb_ru.pack(side="left", padx=15)
self.rb_en.pack(side="left", padx=15)

# ===== SOUND =====
self.btn_sound = Button(
    self, font=("Arial", 22, "bold"),
    width=20, height=2,
    bg="white", fg="black",
    command=self.toggle_sound
)
self.btn_sound.pack(pady=40)

self.update_language()

# ===== LANGUAGE UPDATE =====
def update_language(self):
    lang = "Русский" if self.lang_var.get() == 0 else "English"
    self.controller.language = lang

    is_ru = lang == "Русский"

    self.title.config(text="Настройки" if is_ru else "Settings")
    self.lang_label.config(text="Язык:" if is_ru else "Language:")

```

```

self.btn_sound.config(
    text=("🔊 Звук: Вкл" if self.controller.sound_on else "🔊 Звук: Выкл")
    if is_ru else
    ("🔊 Sound: On" if self.controller.sound_on else "🔊 Sound: Off")
)

# ===== CHANGE LANGUAGE =====
def change_language(self):
    self.update_language()

    save_settings(self.controller.language, self.controller.sound_on)

    for frame in self.controller.frames.values():
        if hasattr(frame, "update_language"):
            frame.update_language()

# ===== SOUND =====
def toggle_sound(self):
    self.controller.sound_on = not self.controller.sound_on

    import winsound
    if self.controller.sound_on:
        self.controller.play_music_loop()
    else:
        winsound.PlaySound(None, winsound.SND_PURGE)

    save_settings(self.controller.language, self.controller.sound_on)
    self.update_language()

```

stats_menu.py

```

import tkinter as tk
from tkinter import Frame, Label, Button
import json
import os

class StatsMenu(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

        # ===== BACK =====
        Button(
            self, text="←", font=("Arial", 24, "bold"),
            bg="white", fg="black", width=3,
            relief="solid", bd=1, cursor="hand2",
            command=lambda: controller.show_frame("MainMenu")
        ).place(x=20, y=20)

        # ===== CENTER =====
        self.center = Frame(self, bg="#0000FF")
        self.center.place(relx=0.5, rely=0.5, anchor="center")

        self.title = Label(self.center, font=("Arial", 36, "bold"),
                           bg="#0000FF", fg="white")
        self.title.pack(pady=(0, 30))

```

```

self.total_games_label = Label(self.center, font=("Arial", 18, "bold"),
                                bg="#0000FF", fg="white")
self.total_games_label.pack()

self.training_label = Label(self.center, font=("Arial", 18, "bold"),
                              bg="#0000FF", fg="white")
self.training_label.pack()

self.best_training_label = Label(self.center, font=("Arial", 18, "bold"),
                                   bg="#0000FF", fg="white")
self.best_training_label.pack(pady=(10, 0))

Frame(self.center, bg="white", height=2).pack(fill="x", padx=250, pady=25)

self.best_title = Label(self.center, font=("Arial", 28, "bold"),
                          bg="#0000FF", fg="white")
self.best_title.pack(pady=(0, 25))

self.games_container = Frame(self.center, bg="#0000FF")
self.games_container.pack()

# ===== CONFIG =====
self.games = {
    "pairs": ("pairs_total", True),
    "sequence": ("sequence_total", False),
    "audio": ("audio_total", False),
    "changes": ("changes_total", False)
}

self.names = {
    "ru": {
        "pairs": "Найди пару",
        "sequence": "Последовательность",
        "audio": "Аудио память",
        "changes": "Что изменилось"
    },
    "en": {
        "pairs": "Find the Pair",
        "sequence": "Sequence",
        "audio": "Audio",
        "changes": "Changes"
    }
}

self.update_language()

# ===== LOAD =====
def load_stats(self):
    if not os.path.exists("stats.json"):
        return {}
    return json.load(open("stats.json", encoding="utf-8"))

# ===== TEXT =====
def diff_text(self, d, ru):
    return {
        "easy": "Легкий" if ru else "Easy",
        "medium": "Средний" if ru else "Medium",
        "hard": "Сложный" if ru else "Hard"
    }[d]

```

```

# ===== UPDATE =====
def update_language(self):
    ru = self.controller.language == "Русский"
    lang = "ru" if ru else "en"

    self.title.config(text="Статистика" if ru else "Statistics")
    self.best_title.config(text="Лучшие результаты" if ru else "Best Scores")

    stats = self.load_stats()

    self.total_games_label.config(
        text=f"{'Всего игр сыграно' if ru else 'Total Games Played'}:
{stats.get('total_games', 0)}"
    )
    self.training_label.config(
        text=f"{'Всего тренировок' if ru else 'Total Training Sessions'}:
{stats.get('training_runs', 0)}"
    )

    best_training_score = stats.get('best_training_score', 0)
    self.best_training_label.config(
        text=f"{'Лучший результат в тренировке' if ru else 'Best Training Score'}:
{best_training_score}"
    )

    for w in self.games_container.winfo_children():
        w.destroy()

    self.games_container.grid_columnconfigure((0, 1), weight=1)

# ===== ГЕНЕРАЦИЯ КАРТОЧЕК =====
for i, (key, (total_key, is_pairs)) in enumerate(self.games.items()):
    row, col = divmod(i, 2)

    played = stats.get(total_key, 0)
    name = self.names[lang][key]

    card = Frame(self.games_container, bg="white", bd=2,
        relief="solid", width=450, height=280)
    card.grid(row=row, column=col, padx=25, pady=15)
    card.grid_propagate(False)

    Label(card, text=f"{name} ({played})",
        font=("Arial", 18, "bold"),
        bg="white", fg="#0000FF").pack(pady=(15, 12))

    for diff in ("easy", "medium", "hard"):
        data = stats.get(key, {}).get(diff, {})

        if not data:
            value = "Нет результатов" if ru else "No score yet"
        else:
            if is_pairs:
                score = data.get("best_score", 0)
                moves = data.get("best_moves", 0)
                value = (
                    f"{'Очки' if ru else 'Score'}: {score} | "
                    f"{'Ходы' if ru else 'Moves'}: {moves}"
                ) if (score or moves) else ("Нет результатов" if ru else "No
score yet")

```

```

        else:
            lvl = data.get("best_level", 0)
            score = data.get("best_score", 0)
            value = (
                f"{'Уровень' if ru else 'Level'}: {lvl} | "
                f"{'Очки' if ru else 'Score'}: {score}"
            ) if (lvl or score) else ("Нет результатов" if ru else "No score
yet")

        row_frame = Frame(card, bg="white")
        row_frame.pack(fill="x", padx=25, pady=6)

        Label(row_frame, text=self.diff_text(diff, ru),
              font=("Arial", 13, "bold"),
              bg="white", width=10, anchor="w").pack(side="left")

        Label(row_frame, text=value,
              font=("Arial", 12),
              bg="white", fg="gray",
              anchor="w").pack(side="left", padx=10)

```

stats_manager.py

```

import json
import os

FILE = "stats.json"
SETTINGS_FILE = "settings.json"

def load():
    return json.load(open(FILE, encoding="utf-8")) if os.path.exists(FILE) else {}

def save(data):
    json.dump(data, open(FILE, "w", encoding="utf-8"), ensure_ascii=False, indent=4)

def update(game, difficulty, win, score, level, moves, language):
    data = load()

    diff_map = {
        "easy": "easy", "medium": "medium", "hard": "hard",
        "Легкий": "easy", "Средний": "medium", "Сложный": "hard",
        "Easy": "easy", "Medium": "medium", "Hard": "hard"
    }
    diff_key = diff_map.get(difficulty, "easy")

    if not data:
        data = {
            "total_games": 0, "training_runs": 0,
            "pairs_total": 0, "sequence_total": 0,
            "audio_total": 0, "changes_total": 0,
            "pairs": {}, "sequence": {}, "audio": {}, "changes": {}
        }

    data.setdefault(game, {})

    if diff_key not in data[game]:

```



```

controller.show_frame("GamesMenu"))
self.back_btn.place(x=20, y=20, width=60, height=60)

# Заголовок
self.title = Label(frame, bg="#0000FF", fg="white", font=("Arial", 50, "bold"))
self.title.pack(pady=(80, 40))

# Кнопки сложности
button_style = {"font": ("Arial", 24, "bold"), "width": 20, "height": 2,
                "bg": "white", "cursor": "hand2"}

self.btn_easy = Button(frame, command=lambda: self.start_game("easy"),
**button_style)
self.btn_easy.pack(pady=15)

self.btn_medium = Button(frame, command=lambda: self.start_game("medium"),
**button_style)
self.btn_medium.pack(pady=15)

self.btn_hard = Button(frame, command=lambda: self.start_game("hard"),
**button_style)
self.btn_hard.pack(pady=15)

self.update_language()

def set_game(self, game_name):
    self.current_game = game_name
    self.update_language()

def start_game(self, difficulty):
    if self.current_game == "pairs":
        game = self.controller.frames["MemoryPairsGame"]
        game.start_game(difficulty)
        self.controller.show_frame("MemoryPairsGame")
    elif self.current_game == "audio":
        game = self.controller.frames["AudioMemoryGame"]
        game.start_game(difficulty)
        self.controller.show_frame("AudioMemoryGame")
    elif self.current_game == "sequence":
        game = self.controller.frames["SequenceMemoryGame"]
        game.start_game(difficulty)
        self.controller.show_frame("SequenceMemoryGame")
    elif self.current_game == 'visual':
        game = self.controller.frames['VisualMemoryGame']
        game.start_game(difficulty)
        self.controller.show_frame('VisualMemoryGame')

def update_language(self):
    if self.controller.language == "Русский":
        self.title.config(text="Выберите сложность")
        self.btn_easy.config(text="Легкий")
        self.btn_medium.config(text="Средний")
        self.btn_hard.config(text="Сложный")
    else:
        self.title.config(text="Select Difficulty")
        self.btn_easy.config(text="Easy")
        self.btn_medium.config(text="Medium")
        self.btn_hard.config(text="Hard")

```

audio_memory_game.py

```
import tkinter as tk
from tkinter import Frame, Label, Button
import random
from functools import partial
from stats_manager import update
import winsound
import os

class AudioMemoryGame(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller
        self.current_difficulty = "easy"

        self.sequence = []
        self.user_sequence = []
        self.lock = True

        self.main_container = Frame(self, bg="#0000FF")
        self.main_container.pack(expand=True, fill="both")

        self.overlay = Frame(self.main_container, bg="#0000FF")

        # ===== TOP =====
        top_frame = Frame(self.main_container, bg="#0000FF")
        top_frame.pack(fill="x", pady=(30, 10), padx=40)

        self.pause_btn = Button(
            top_frame, text="II",
            font=("Arial", 18, "bold"),
            bg="white", width=3,
            command=self.show_pause
        )
        self.pause_btn.pack(side="left")

        self.title = Label(
            top_frame, bg="#0000FF", fg="white",
            font=("Arial", 36, "bold")
        )
        self.title.pack(side="left", expand=True)

        # ===== STATS =====
        stats_frame = Frame(self.main_container, bg="#0000FF")
        stats_frame.pack(pady=(10, 30))

        self.level_label = Label(stats_frame, bg="white", font=("Arial", 16, "bold"),
width=12)
        self.level_label.pack(side="left", padx=20)

        self.points_label = Label(stats_frame, bg="white", font=("Arial", 16, "bold"),
width=12)
        self.points_label.pack(side="left", padx=20)

        # ===== GAME =====
        self.center_frame = Frame(self.main_container, bg="#0000FF")
        self.center_frame.pack(expand=True, fill="both")
```

```

self.game_frame = Frame(self.center_frame, bg="#0000FF")
self.game_frame.place(relx=0.5, rely=0.5, anchor="center")

self.update_language()

self.training_mode = False
self.training_callback = None

# ===== LANGUAGE =====
def update_language(self):
    if getattr(self.controller, "language", "EN") == "Русский":
        self.title.config(text="Аудио память")
        self.level_text = "Уровень"
        self.score_text = "Очки"
        self.pause_text = "ПАУЗА"
        self.resume_text = "Продолжить"
        self.menu_text = "Главное меню"
        self.lose_text = "Вы ошиблись!"
        self.lang_code = "ru"
    else:
        self.title.config(text="Audio Memory")
        self.level_text = "Level"
        self.score_text = "Score"
        self.pause_text = "PAUSE"
        self.resume_text = "Resume"
        self.menu_text = "Main Menu"
        self.lose_text = "Wrong!"
        self.lang_code = "en"

# ===== DIFFICULTY =====
def get_difficulty_text(self):
    return {
        "ru": {"easy": "Легкий", "medium": "Средний", "hard": "Сложный"},
        "en": {"easy": "Easy", "medium": "Medium", "hard": "Hard"}
    }[self.lang_code][self.current_difficulty]

# ===== SOUND =====
def play_sound(self, idx):
    path = os.path.join("sounds", self.lang_code, f"{idx + 1}.wav")
    if os.path.isfile(path):
        winsound.PlaySound(path, winsound.SND_FILENAME | winsound.SND_ASYNC)

# ===== START =====
def start_game(self, difficulty):
    self.controller.in_game = True
    winsound.PlaySound(None, winsound.SND_PURGE)
    self.hide_overlay()

    self.current_difficulty = difficulty
    self.difficulty_multiplier = {"easy": 1, "medium": 2, "hard": 3}[difficulty]

    rows, cols = {"easy": (2, 2), "medium": (3, 2), "hard": (3, 3)}[difficulty]

    self.current_level = 1
    self.correct_sequences = 0

    self.sequence = []
    self.user_sequence = []
    self.lock = True

```

```

self.update_level()
self.update_points()

for w in self.game_frame.wininfo_children():
    w.destroy()

self.buttons = []
for i in range(rows * cols):
    btn = Button(
        self.game_frame,
        text=str(i + 1),
        width=6, height=3,
        font=("Arial", 24, "bold"),
        command=partial(self.button_pressed, i)
    )
    btn.grid(row=i // cols, column=i % cols, padx=10, pady=10)
    self.buttons.append(btn)

self.after(500, self.add_to_sequence)

# ===== GAME =====
def add_to_sequence(self):
    self.sequence.append(random.randint(0, len(self.buttons) - 1))
    self.user_sequence = []
    self.play_sequence()

def play_sequence(self):
    self.lock = True
    for i, idx in enumerate(self.sequence):
        self.after(800 * (i + 1), lambda x=idx: self.play_sound(x))
    self.after(800 * (len(self.sequence) + 1), self.unlock)

def unlock(self):
    self.lock = False

def button_pressed(self, idx):
    if self.lock or not self.sequence:
        return

    self.user_sequence.append(idx)

    btn = self.buttons[idx]
    btn.config(bg="#7CFC00")
    self.after(200, lambda: btn.config(bg="white"))

    pos = len(self.user_sequence) - 1

    if self.user_sequence[-1] != self.sequence[pos]:
        btn.config(bg="#ff9999")
        self.after(200, lambda: btn.config(bg="white"))
        return self.end_game()

    if len(self.user_sequence) == len(self.sequence):
        self.correct_sequences += 1
        self.current_level += 1

        self.update_level()
        self.update_points()

```

```

        self.after(600, self.add_to_sequence)

# ===== SCORE =====
def calculate_score(self):
    return max(0, self.correct_sequences * 100 * self.difficulty_multiplier)

def get_current_score(self):
    return self.correct_sequences * 100 * self.difficulty_multiplier

def end_game(self):
    self.final_score = self.calculate_score()

    update(
        "audio",
        self.current_difficulty,
        False,
        self.final_score,
        self.current_level,
        0,
        self.controller.language
    )

    if getattr(self, "training_mode", False) and self.training_callback:
        cb = self.training_callback
        self.training_callback = None
        self.after(300, lambda: cb(self.final_score, self.current_level))
        return

    self.show_fail_screen()

# ===== UI =====
def update_level(self):
    self.level_label.config(text=f"{self.level_text}: {self.current_level}")

def update_points(self):
    self.points_label.config(text=f"{self.score_text}: {self.get_current_score()}")

# ===== PAUSE =====
def show_pause(self):
    self.overlay.place(relx=0, rely=0, relwidth=1, relheight=1)
    self.overlay.lift()

    for w in self.overlay.winfo_children():
        w.destroy()

    box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
    box.place(relx=0.5, rely=0.5, anchor="center")
    box.pack_propagate(False)

    Label(box, text=self.pause_text,
          font=("Arial", 42, "bold"),
          bg="#0000FF", fg="white").pack(pady=40)

    Button(box, text=self.resume_text,
          font=("Arial", 18, "bold"),
          bg="white", width=18,
          command=self.hide_overlay).pack(pady=15)

    Button(box, text=self.menu_text,
          font=("Arial", 18, "bold"),

```

```

        bg="white", width=18,
        command=self.go_to_main_menu).pack(pady=15)

# ===== FAIL =====
def show_fail_screen(self):
    self.overlay.place(relx=0, rely=0, relwidth=1, relheight=1)
    self.overlay.lift()

    for w in self.overlay.winfo_children():
        w.destroy()

    box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
    box.place(relx=0.5, rely=0.5, anchor="center")
    box.pack_propagate(False)

    Label(box, text=self.title.cget("text"),
          font=("Arial", 36, "bold"),
          bg="#0000FF", fg="white").pack(pady=(20, 5))

    Label(box, text=self.get_difficulty_text(),
          font=("Arial", 24, "bold"),
          bg="#0000FF", fg="white").pack(pady=(0, 20))

    Label(box, text=f"{self.score_text}: {self.final_score}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

    Label(box, text=f"{self.level_text}: {self.current_level}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

    Button(box, text=self.menu_text,
           font=("Arial", 18, "bold"),
           bg="white", width=18,
           command=self.go_to_main_menu).pack(pady=20)

def hide_overlay(self):
    self.overlay.place_forget()

def go_to_main_menu(self):
    self.hide_overlay()
    self.controller.in_game = False
    self.controller.play_music_loop()
    self.controller.show_frame("MainMenu")

```

memory_pairs_game.py

```

import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random

class MemoryPairsGame(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

        self.main_container = Frame(self, bg="#0000FF")

```

```

self.main_container.pack(expand=True, fill="both")

# ===== TOP =====
top_frame = Frame(self.main_container, bg="#0000FF")
top_frame.pack(fill="x", pady=(30, 10), padx=40)

Button(top_frame, text="II ",
        font=("Arial", 18, "bold"),
        bg="white", width=3,
        command=self.show_pause_menu).pack(side="left")

self.game_title = Label(top_frame,
                        bg="#0000FF",
                        fg="white",
                        font=("Arial", 36, "bold"))
self.game_title.pack(side="left", expand=True)

# ===== STATS =====
stats_frame = Frame(self.main_container, bg="#0000FF")
stats_frame.pack(pady=(10, 30))

self.moves_label = Label(stats_frame, bg="white",
                        font=("Arial", 16, "bold"))
self.moves_label.pack(side="left", padx=20)

self.points_label = Label(stats_frame, bg="white",
                        font=("Arial", 16, "bold"))
self.points_label.pack(side="left", padx=20)

# ===== GAME =====
self.center_frame = Frame(self.main_container, bg="#0000FF")
self.center_frame.pack(expand=True, fill="both")

self.game_frame = Frame(self.center_frame, bg="#0000FF")
self.game_frame.place(relx=0.5, rely=0.5, anchor="center")

self.overlay = Frame(self.main_container, bg="#0000FF")

self.update_language()

self.training_mode = False
self.training_callback = None

# ===== LANGUAGE =====
def update_language(self):
    if self.controller.language == "Русский":
        self.game_title.config(text="Найди пару")
        self.moves_text = "Ходы"
        self.score_text = "Очки"
        self.pause_text = "ПАУЗА"
        self.resume_text = "Продолжить"
        self.menu_text = "Главное меню"
        self.win_text = "ПОБЕДА!"
    else:
        self.game_title.config(text="Find the Pair")
        self.moves_text = "Moves"
        self.score_text = "Score"
        self.pause_text = "PAUSE"
        self.resume_text = "Resume"
        self.menu_text = "Main Menu"

```



```

        self.win_text = "YOU WON!"

    def get_difficulty_text(self):
        if self.controller.language == "Русский":
            return {"easy": "Легкий", "medium": "Средний", "hard":
"Сложный"}[self.current_difficulty]
        else:
            return {"easy": "Easy", "medium": "Medium", "hard":
"Hard"}[self.current_difficulty]

# ===== START =====
    def start_game(self, difficulty):
        self.controller.in_game = True

        import winsound
        winsound.PlaySound(None, winsound.SND_PURGE)
        self.hide_overlay()

        self.current_difficulty = difficulty

        self.difficulty_multiplier = {"easy": 1, "medium": 2, "hard": 3}[difficulty]

        rows, cols = {"easy": (3, 2), "medium": (4, 4), "hard": (5, 6)}[difficulty]

        total = rows * cols
        self.total_pairs = total // 2
        self.min_moves = self.total_pairs * 2

        self.found_pairs = 0
        self.moves = 0

        self.update_moves()
        self.update_points()

        for w in self.game_frame.wininfo_children():
            w.destroy()

        numbers = list(range(self.total_pairs)) * 2
        random.shuffle(numbers)

        self.cards = numbers
        self.buttons = []
        self.first = None
        self.second = None
        self.lock = False

        index = 0
        for r in range(rows):
            for c in range(cols):
                btn = Button(
                    self.game_frame,
                    text="?",
                    width=6,
                    height=3,
                    font=("Arial", 20, "bold"),
                    bg="white",
                    command=lambda i=index: self.flip(i)
                )
                btn.grid(row=r, column=c, padx=10, pady=10)
                self.buttons.append(btn)

```

```

        index += 1

# ===== GAME =====
def flip(self, index):
    if self.lock:
        return

    btn = self.buttons[index]
    if btn["text"] != "?":
        return

    btn.config(text=str(self.cards[index]), bg="#ADD8E6")

    if self.first is None:
        self.first = index
        return

    self.second = index
    self.lock = True
    self.moves += 1

    self.update_moves()
    self.update_points()

    self.after(500, self.check)

def check(self):
    if self.cards[self.first] == self.cards[self.second]:
        for i in [self.first, self.second]:
            self.buttons[i].config(bg="#7CFC00", state="disabled")

        self.found_pairs += 1
        self.update_points()

        if self.found_pairs == self.total_pairs:
            self.end_game()
    else:
        for i in [self.first, self.second]:
            self.buttons[i].config(bg="#ff9999")

        def reset():
            self.buttons[self.first].config(text="?", bg="white")
            self.buttons[self.second].config(text="?", bg="white")
            self.first = self.second = None
            self.lock = False

        self.after(400, reset)
        return

    self.first = None
    self.second = None
    self.lock = False

# ===== SCORE =====
def calculate_score(self):
    pairs_bonus = self.found_pairs * 100
    extra_moves = max(0, self.moves - self.min_moves)
    penalty = extra_moves * 10
    return max(0, (pairs_bonus * self.difficulty_multiplier) - penalty)

```

```

def get_current_score(self):
    pairs_bonus = self.found_pairs * 100
    extra_moves = max(0, self.moves - self.min_moves)
    penalty = extra_moves * 10
    return max(0, (pairs_bonus * self.difficulty_multiplier) - penalty)

def end_game(self):
    self.final_score = self.calculate_score()

    update("pairs", self.current_difficulty, True,
           self.final_score, 0, self.moves,
           self.controller.language)

    #  ИСПРАВЛЕНО ЗДЕСЬ
    if self.training_mode and self.training_callback:
        callback = self.training_callback
        self.training_callback = None
        self.after(300, lambda: callback(self.final_score, self.moves))
        return

    self.show_end()

# ===== UI =====
def update_moves(self):
    self.moves_label.config(text=f"{self.moves_text}: {self.moves}")

def update_points(self):
    self.points_label.config(text=f"{self.score_text}: {self.get_current_score()}")

# ===== OVERLAY =====
def show_end(self):
    self.overlay.place(relwidth=1, relheight=1)
    self.overlay.lift()

    for w in self.overlay.winfo_children():
        w.destroy()

    box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
    box.place(relx=0.5, rely=0.5, anchor="center")
    box.pack_propagate(False)

    Label(box, text=self.game_title.cget("text"),
          font=("Arial", 36, "bold"),
          bg="#0000FF", fg="white").pack(pady=(20, 5))

    Label(box, text=self.get_difficulty_text(),
          font=("Arial", 24, "bold"),
          bg="#0000FF", fg="white").pack(pady=(0, 20))

    Label(box, text=f"{self.score_text}: {self.final_score}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

    Label(box, text=f"{self.moves_text}: {self.moves}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

    Button(box, text=self.menu_text,
           font=("Arial", 18, "bold"),
           bg="white", width=18,

```

```

        command=self.go_to_main_menu).pack(pady=20)

def show_pause_menu(self):
    self.show_overlay(self.pause_text, True)

def show_overlay(self, title, resume=False):
    for w in self.overlay.winfo_children():
        w.destroy()

    self.overlay.place(relwidth=1, relheight=1)
    self.overlay.lift()

    box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
    box.place(relx=0.5, rely=0.5, anchor="center")
    box.pack_propagate(False)

    Label(box, text=title,
          font=("Arial", 42, "bold"),
          bg="#0000FF", fg="white").pack(pady=40)

    if resume:
        Button(box, text=self.resume_text,
              font=("Arial", 18, "bold"),
              bg="white", width=18,
              command=self.hide_overlay).pack(pady=15)

        Button(box, text=self.menu_text,
              font=("Arial", 18, "bold"),
              bg="white", width=18,
              command=self.go_to_main_menu).pack(pady=15)

def hide_overlay(self):
    self.overlay.place_forget()

def go_to_main_menu(self):
    self.hide_overlay()
    self.controller.in_game = False
    self.controller.play_music_loop()
    self.controller.show_frame("MainMenu")

```

remember_sequence_game.py

```

port tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random

class SequenceMemoryGame(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

        self.main_container = Frame(self, bg="#0000FF")
        self.main_container.pack(expand=True, fill="both")

        top_frame = Frame(self.main_container, bg="#0000FF")
        top_frame.pack(fill="x", pady=(30, 10), padx=40)

```

```

Button(top_frame, text="II", font=("Arial", 18, "bold"),
       bg="white", width=3, command=self.pause).pack(side="left")

self.game_title = Label(top_frame, bg="#0000FF", fg="white",
                        font=("Arial", 36, "bold"))
self.game_title.pack(side="left", expand=True)

stats = Frame(self.main_container, bg="#0000FF")
stats.pack(pady=20)

self.level_label = Label(stats, bg="white", font=("Arial", 16, "bold"), width=12)
self.level_label.pack(side="left", padx=20)

self.points_label = Label(stats, bg="white", font=("Arial", 16, "bold"),
width=12)
self.points_label.pack(side="left", padx=20)

self.game_frame = Frame(self.main_container, bg="#0000FF")
self.game_frame.pack(expand=True)

self.overlay = Frame(self.main_container, bg="#0000FF")

self.update_language()
self.training_mode = False
self.training_callback = None

def update_language(self):
    if self.controller.language == "Русский":
        self.game_title.config(text="Запомни последовательность")
        self.level_text = "Уровень"
        self.points_text = "Очки"
        self.pause_text = "ПАУЗА"
        self.resume_text = "Продолжить"
        self.menu_text = "Главное меню"
    else:
        self.game_title.config(text="Remember Sequence")
        self.level_text = "Level"
        self.points_text = "Score"
        self.pause_text = "PAUSE"
        self.resume_text = "Resume"
        self.menu_text = "Main Menu"

def get_difficulty_text(self):
    return {
        "easy": {"ru": "Легкий", "en": "Easy"},
        "medium": {"ru": "Средний", "en": "Medium"},
        "hard": {"ru": "Сложный", "en": "Hard"}
    }[self.current_difficulty][self.controller.language == "Русский" else
"en"]

def start_game(self, difficulty):
    self.controller.in_game = True
    import winsound
    winsound.PlaySound(None, winsound.SND_PURGE)
    self.hide_overlay()

    self.current_difficulty = difficulty
    self.difficulty_multiplier = {"easy": 1, "medium": 2, "hard": 3}[difficulty]

    sizes = {"easy": (2, 2), "medium": (2, 3), "hard": (3, 3)}

```

```

rows, cols = sizes[difficulty]

for w in self.game_frame.winfo_children():
    w.destroy()

self.buttons = []
index = 0

for r in range(rows):
    for c in range(cols):
        btn = Button(self.game_frame, text=str(index + 1),
                      width=6, height=3, font=("Arial", 24, "bold"),
                      command=lambda i=index: self.press(i))
        btn.grid(row=r, column=c, padx=10, pady=10)
        self.buttons.append(btn)
        index += 1

self.sequence = []
self.user_sequence = []
self.correct_sequences = 0
self.current_level = 1

self.update_stats()
self.lock = True
self.after(500, self.add)

def add(self):
    self.sequence.append(random.randint(0, len(self.buttons) - 1))
    self.user_sequence = []
    self.play()

def play(self):
    self.lock = True
    for i, idx in enumerate(self.sequence):
        self.after(600 * (i + 1), lambda b=idx: self.flash(b))
    self.after(600 * (len(self.sequence) + 1),
              lambda: setattr(self, "lock", False))

def flash(self, idx):
    b = self.buttons[idx]
    b.config(bg="yellow")
    self.after(300, lambda: b.config(bg="white"))

def press(self, idx):
    if self.lock or not self.sequence:
        return

    btn = self.buttons[idx]
    btn.config(bg="#7CFC00")
    self.after(200, lambda: btn.config(bg="white"))

    self.user_sequence.append(idx)
    pos = len(self.user_sequence) - 1

    if pos >= len(self.sequence) or self.user_sequence[-1] != self.sequence[pos]:
        btn.config(bg="#ff9999")
        self.after(200, lambda: btn.config(bg="white"))
        self.end_game()
        return

```

```

        if len(self.user_sequence) == len(self.sequence):
            self.correct_sequences += 1
            self.current_level += 1
            self.update_stats()
            self.after(600, self.add)

    def calculate_score(self):
        return max(0, self.correct_sequences * 100 * self.difficulty_multiplier)

    def get_current_score(self):
        return max(0, self.correct_sequences * 100 * self.difficulty_multiplier)

    def end_game(self):
        self.final_score = self.calculate_score()

        update("sequence", self.current_difficulty, False,
              self.final_score, self.current_level, 0,
              self.controller.language)

        if getattr(self, "training_mode", False) and self.training_callback:
            cb = self.training_callback
            self.training_callback = None
            self.after(300, lambda: cb(self.final_score, self.current_level))
            return

        self.show_overlay()

    def update_stats(self):
        self.level_label.config(text=f"{self.level_text}: {self.current_level}")
        self.points_label.config(text=f"{self.points_text}: {self.get_current_score()}")

    def show_overlay(self, resume=False):
        for w in self.overlay.winfo_children():
            w.destroy()

        self.overlay.place(relwidth=1, relheight=1)
        self.overlay.lift()

        box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
        box.place(relx=0.5, rely=0.5, anchor="center")
        box.pack_propagate(False)

        Label(box, text=self.game_title.cget("text"),
              font=("Arial", 34, "bold"),
              bg="#0000FF", fg="white").pack(pady=(20, 5))

        Label(box, text=self.get_difficulty_text(),
              font=("Arial", 24, "bold"),
              bg="#0000FF", fg="white").pack(pady=(0, 20))

        if not resume:
            Label(box, text=f"{self.points_text}: {self.final_score}",
                  font=("Arial", 20, "bold"),
                  bg="#0000FF", fg="white").pack(pady=10)

            Label(box, text=f"{self.level_text}: {self.current_level}",
                  font=("Arial", 20, "bold"),
                  bg="#0000FF", fg="white").pack(pady=10)

        if resume:

```

```

        Button(box, text=self.resume_text,
               font=("Arial", 18, "bold"),
               bg="white", width=18,
               command=self.hide_overlay).pack(pady=15)

        Button(box, text=self.menu_text,
               font=("Arial", 18, "bold"),
               bg="white", width=18,
               command=self.go_to_main_menu).pack(pady=15)

    def pause(self):
        self.show_overlay(True)

    def hide_overlay(self):
        self.overlay.place_forget()

    def go_to_main_menu(self):
        self.hide_overlay()
        self.controller.in_game = False
        self.controller.play_music_loop()
        self.controller.show_frame("MainMenu")

```

visual_memory_games.py

```

import tkinter as tk
from tkinter import Frame, Label, Button
from stats_manager import update
import random
import winsound

class VisualMemoryGame(Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, bg="#0000FF")
        self.controller = controller

        self.main_container = Frame(self, bg="#0000FF")
        self.main_container.pack(expand=True, fill="both")

        top = Frame(self.main_container, bg="#0000FF")
        top.pack(fill="x", pady=(30, 10), padx=40)

        Button(top, text="II", font=("Arial", 18, "bold"),
              bg="white", width=3, command=self.pause).pack(side="left")

        self.title = Label(top, bg="#0000FF", fg="white",
                          font=("Arial", 36, "bold"))
        self.title.pack(side="left", expand=True)

        stats = Frame(self.main_container, bg="#0000FF")
        stats.pack(pady=20)

        self.level_label = Label(stats, bg="white", font=("Arial", 16, "bold"), width=12)
        self.level_label.pack(side="left", padx=20)

        self.points_label = Label(stats, bg="white", font=("Arial", 16, "bold"),
                                width=12)
        self.points_label.pack(side="left", padx=20)

```



```

self.timer_label = Label(self.main_container, bg="white",
                          font=("Arial", 16, "bold"))
self.timer_label.pack(pady=10)

self.game_frame = Frame(self.main_container, bg="#0000FF")
self.game_frame.pack(expand=True)

self.overlay = Frame(self, bg="#0000FF")

self.update_language()
self.training_mode = False
self.training_callback = None

self.current_difficulty = None
self.difficulty_multiplier = None
self.show_time = None
self.current_level = None
self.correct_rounds = None
self.values = None
self.to_change = None
self.time_left = None
self.lock = None
self.user = None

def update_language(self):
    if self.controller.language == "Русский":
        self.title.config(text="Что изменилось?")
        self.level_text = "Уровень"
        self.points_text = "Очки"
        self.time_text = "Время"
        self.pause_text = "ПАУЗА"
        self.resume_text = "Продолжить"
        self.menu_text = "Главное меню"
    else:
        self.title.config(text="What Changed?")
        self.level_text = "Level"
        self.points_text = "Score"
        self.time_text = "Time"
        self.pause_text = "PAUSE"
        self.resume_text = "Resume"
        self.menu_text = "Main Menu"

def get_difficulty_text(self):
    return {
        "easy": {"ru": "Легкий", "en": "Easy"},
        "medium": {"ru": "Средний", "en": "Medium"},
        "hard": {"ru": "Сложный", "en": "Hard"}
    }[self.current_difficulty][self.controller.language == "Русский" else
"en"]

def start_game(self, difficulty):
    self.hide_overlay()
    self.lock = True

    self.controller.in_game = True
    winsound.PlaySound(None, winsound.SND_PURGE)

    self.current_difficulty = difficulty
    self.difficulty_multiplier = {"easy": 1, "medium": 2, "hard": 3}[difficulty]

```

```

self.times = {"easy": 5000, "medium": 4000, "hard": 3000}
self.show_time = self.times[difficulty]

for w in self.game_frame.winfo_children():
    w.destroy()

self.buttons = []
for i in range(6):
    btn = Button(self.game_frame,
                  width=8, height=4,
                  font=("Arial", 24, "bold"),
                  command=lambda idx=i: self.press(idx))
    btn.grid(row=i // 3, column=i % 3, padx=15, pady=15)
    self.buttons.append(btn)

self.current_level = 1
self.correct_rounds = 0

self.update_stats()
self.new_round()

def new_round(self):
    self.lock = True
    self.user = []

    self.values = list(range(1, 7))
    random.shuffle(self.values)

    for i, b in enumerate(self.buttons):
        b.config(text=str(self.values[i]), bg="white")

    self.to_change = random.sample(range(6), 2)

    self.time_left = self.show_time // 1000
    self.update_timer()

    self.after(self.show_time, self.change_cards)

def update_timer(self):
    self.timer_label.config(text=f"{self.time_text}: {self.time_left}")
    if self.time_left > 0:
        self.time_left -= 1
        self.after(1000, self.update_timer)

def change_cards(self):
    self.fade_step(0)

def fade_step(self, step):
    colors = ["#FFFFFF", "#DDDDDD", "#AAAAAA", "#777777"]

    if step < len(colors):
        for b in self.buttons:
            b.config(bg=colors[step])
        self.after(100, lambda: self.fade_step(step + 1))
    else:
        self.swap_cards()

def swap_cards(self):
    i1, i2 = self.to_change
    self.values[i1], self.values[i2] = self.values[i2], self.values[i1]

```

```

        for i, b in enumerate(self.buttons):
            b.config(text=str(self.values[i]), bg="white")

        self.lock = False

    def press(self, idx):
        if self.lock:
            return

        if idx in self.to_change:
            if idx not in self.user:
                self.user.append(idx)
                self.buttons[idx].config(bg="green")

            if len(self.user) == 2:
                self.correct_rounds += 1
                self.current_level += 1
                self.update_stats()
                self.after(800, self.new_round)
        else:
            self.end_game()

    def calculate_score(self):
        return max(0, self.correct_rounds * 100 * self.difficulty_multiplier)

    def get_current_score(self):
        return max(0, self.correct_rounds * 100 * self.difficulty_multiplier)

    def end_game(self):
        self.final_score = self.calculate_score()

        update("changes", self.current_difficulty, False,
              self.final_score, self.current_level, 0,
              self.controller.language)

        if getattr(self, "training_mode", False) and self.training_callback:
            cb = self.training_callback
            self.training_callback = None
            self.after(300, lambda: cb(self.final_score, self.current_level))
            return

        self.show_overlay()

    def update_stats(self):
        self.level_label.config(text=f"{self.level_text}: {self.current_level}")
        self.points_label.config(text=f"{self.points_text}: {self.get_current_score()}")

    def show_overlay(self, resume=False):
        for w in self.overlay.winfo_children():
            w.destroy()

        self.overlay.place(relwidth=1, relheight=1)
        self.overlay.lift()

        box = Frame(self.overlay, bg="#0000FF", width=700, height=450)
        box.place(relx=0.5, rely=0.5, anchor="center")
        box.pack_propagate(False)

        Label(box, text=self.title.cget("text"),

```

```

        font=("Arial", 36, "bold"),
        bg="#0000FF", fg="white").pack(pady=(20, 5))

Label(box, text=self.get_difficulty_text(),
      font=("Arial", 24, "bold"),
      bg="#0000FF", fg="white").pack(pady=(0, 20))

if not resume:
    Label(box, text=f"{self.points_text}: {self.final_score}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

    Label(box, text=f"{self.level_text}: {self.current_level}",
          font=("Arial", 20, "bold"),
          bg="#0000FF", fg="white").pack(pady=10)

if resume:
    Button(box, text=self.resume_text,
           font=("Arial", 18, "bold"),
           bg="white", width=18,
           command=self.hide_overlay).pack(pady=15)

    Button(box, text=self.menu_text,
           font=("Arial", 18, "bold"),
           bg="white", width=18,
           command=self.go_to_main_menu).pack(pady=15)

def go_to_main_menu(self):
    self.hide_overlay()
    self.controller.in_game = False
    self.controller.play_music_loop()
    self.controller.show_frame("MainMenu")

def hide_overlay(self):
    self.overlay.place_forget()

def pause(self):
    self.show_overlay(True)

```

Список использованных источников

1. Преимущества языка Python // URL - <https://www.hocktraining.com/blog/preimuschestva-yazyka-python> (Дата обращения 31.01.2026)
2. Что такое PyCharm – основные возможности и функции // URL - <https://skyeng.ru/magazine/cto-takoe-pycharm/> (Дата обращения 31.01.2026)
3. ПостНаука – Как работает память человека // URL:(Дата обращения 24.01.2026)
4. Виды памяти у человека в психологии // URL: <https://nadpo.ru/academy/blog/vidy-pamyati-u-cheloveka-v-psihologii/> (Дата обращения 24.01.2026)
5. Учёные выяснили, что использование гаджетов не ослабляет человеческую память // URL: <https://skillbox.ru/media/education/uchyenyevyyasnili-cto-ispolzovanie-gadzhetov-ne-oslablyaet-chelovecheskuyu-pamyat/> (Дата обращения 24.01.2026)
6. Влияние видеоигр на когнитивные способности: исследования // URL: <https://4brain.ru/blog/vliyanie-videoigr/> (Дата обращения 24.01.2026)